

교과목 3 : 초거대언어모델(LLM)

단원 4 : 프롬프트 엔지니어링

DAY7. LangGraph 기반 멀티 에이전트 구조와 응용

목 차

1. Role based agent		3
2. AI team		26
3. Project design		49

1. Role based agent

무엇을 할지와 어떻게 할지 나누기

이전 장에서 Supervisor 는 질문을 한 명에게 위임했습니다. 하지만 현실 업무는 보통 여러 단계로 쪼개야 합니다.

Planner(설계자) 가 "무엇을 할지"를 정해 할 일 목록을 만들고, **Worker(실행자)** 가 각 항목을 "어떻게 할지" 처리합니다. 백엔드의 "작업 큐 + 워커" 패턴과 같은 구조죠.

1. 한 번에 다 시키면 어설퍼진다

마케팅 캠페인 하나를 준비한다고 합시다. 필요한 일은:

1. 타겟 고객 분석
2. 광고 카피 작성
3. 채널/예산 배분 제안
4. 셋을 합쳐 실행안으로 정리

이걸 한 번의 LLM 호출에 "캠페인 다 준비해줘" 라고 던지면?

- 일부를 빠뜨림 (카피만 쓰고 예산 배분은 깜빡)
- 각 부분의 품질이 들쭉날쭉 (한 번에 다 하려니 어설편)
- "왜 이렇게 했는지" 단계가 안 보임 (한 덩어리 출력)

백엔드로 치면 거대한 트랜잭션을 함수 하나에 다 욕여넣은 셈입니다. 한 번에 다 하려다 다 어설퍼지죠.

2. "무엇을"과 "어떻게"를 나누기

해결책은 일을 두 역할로 나누는 것입니다.

[Planner — 무엇을 할지]

캠페인 브리프를 보고 → "타겟 분석 / 카피 작성 / 예산 배분" 할 일 목록 생성
(실행은 안 함, 계획만)

[Worker — 어떻게 할지]

할 일 하나씩 받아 → 각각 구체적으로 실행
(계획은 안 함, 실행만)

캠페인 1건 이 세 단계를 거칩니다.

1. [Planner] "어떤 하위 작업이 필요한가?" → [타깃분석, 카피작성, 예산배분]
2. [Worker] (각 작업을 하나씩) 작업1 실행 → 작업2 실행 → 작업3 실행
3. [취합] 세 산출물을 모아 최종 실행안

각자 자기 일에만 집중하니 품질이 올라갑니다. Planner는 빠짐없이 쪼개는 데, Worker는 한 작업을 깊이 하는 데 몰두하죠.

3. 왜 나누면 좋은가

[한 LLM이 다 하기]	[Planner / Worker 분리]
"계획도 세우고 실행도 해"	→ Planner는 계획만, Worker는 실행만
- 둘 다 어설피름	- 각자 잘하는 일에 몰두 → 품질 ↑
- 빠뜨리기 쉬움	- Planner가 빠짐없이 쪼갬
- 단계 안 보임	- 작업 단위로 추적·재시도 가능

이건 앞 장의 **역할 분리**의 연장선입니다. 앞 장은 "추천 vs 정책"처럼 **수평**으로 나뉘었다면, 이번 장은 "계획 → 실행"처럼 **수직(계층)**으로 나눕니다. 둘 다 "한 명에게 다 시키지 말고 역할을 나눠라"는 같은 정신이죠.

4. 백엔드의 "작업 큐 + 워커" 패턴

이 구조는 백엔드 개발자에게 익숙합니다.

[작업 큐 + 워커 패턴]
생산자(Planner): 할 일을 큐에 넣음 → [작업1, 작업2, 작업3]
소비자(Worker): 큐에서 하나씩 꺼내 처리
취합: 처리 결과를 모음

- 작업이 독립적 → 병렬 처리 가능 (Worker 여러 개)
- 한 작업 실패 → 그 작업만 재시도 (07번)
- 작업 추가 쉬움 → 큐에 넣기만

에이전트의 Planner/Worker가 정확히 이 패턴입니다. Planner가 작업을 만들고, Worker가 꺼내 처리하죠. 익숙한 백엔드 패턴을 LLM에 적용한 겁니다.

5. 우리가 만들 것 — 캠페인 기획 자동화

이번 강에서는 `marketing_brief.csv`의 캠페인 1건으로 기획 자동화 파이프라인을 만듭니다.

`marketing_brief.csv` (campaign_id, title, category, target, budget, key_offer)에서 캠페인 1건(예: CMP02 신학기 전자기기 프로모션)을 선택해 처리합니다.

1. **[Planner]** 하위 작업 3~4개로 분해 (구조화 출력 JSON)
2. **[Worker]** 각 작업을 하나씩 실행 (타겟 분석·카피·예산)
3. **[취합]** 마크다운 캠페인 실행안

그리고 신규 항목으로 **Worker 실패 처리·재시도(07번)**를 더해, 한 작업이 실패해도 나머지는 살리는 견고한 파이프라인을 만듭니다.

6. 이번 강에서 배울 것

1. Planner/Worker 협업 구조 (02번)
2. Supervisor vs Planner/Worker 비교 (03번)
3. Planner — 작업을 JSON으로 분해 (04번)
4. Worker — 한 작업씩 실행 (05번)
5. 오케스트레이터 — 계획→실행→취합 (06번)
6. [신규] Worker 실패 처리와 재시도 (07번)

Planner / Worker 협업 구조

Planner 는 전략가(무엇을 할지), Worker 는 실무자(어떻게 할지)입니다. Planner 가 작업을 **JSON 리스트**로 쪼개면 Worker 가 **한 작업씩** 충실히 수행하고, 마지막에 산출물을 **취합**합니다. 에이전트 간 데이터는 **구조화 출력(JSON)**으로 안전하게 주고받습니다.

1. 세 단계 구조

캠페인 1건 이 세 단계를 거칩니다.

1. **[Planner]** 전략가 — 실행은 안 하고 '무엇을 할지' 결정 → ["타겟 분석", "카피 작성", "예산 배분"] (구조화 출력 JSON)
2. **[Worker]** 실무자 — 받은 '한 작업만' 충실히 수행(반복): 작업1 → 산출물1, 작업2 → 산출물2, 작업3 → 산출물3
3. **[취합]** Worker 산출물을 모아 최종 실행안으로 정리

역할	하는 일	특징
Planner	무엇을 할지 결정 → 하위 작업 리스트	실행 안 함, 계획만
Worker	받은 한 작업만 수행	재사용·병렬화 가능
취합	산출물을 모아 정리	최종 결과 생성

2. Planner — 전략가

Planner는 실행하지 않습니다. 오직 "무엇을 할지"만 정하죠.

```
PLANNER_SYSTEM = (
    "너는 마케팅 캠페인 '기획 설계자(Planner)'다. "
    "주어진 캠페인 브리프를 보고, 실행에 필요한 하위 작업을 3~4개로 쪼개라. "
    "작업은 서로 겹치지 않게, 한 문장 명령형으로 작성하라. 실행은 하지 마라." # ← 실행 금지!
)
```

프롬프트의 "실행은 하지 마라"가 핵심입니다. Planner가 욕심내서 직접 카피까지 쓰면 안 됩니다 — 쪼개기만 하고 실행은 Worker에게 넘겨야 역할 분리가 유지되죠. 또 "서로 겹치지 않게"로 작업 중복을 막고, "한 문장 명령형"으로 Worker가 받기 좋은 형태를 강제합니다.

3. Worker — 실무자

Worker는 받은 한 작업만 깊이 수행합니다.

```
WORKER_SYSTEM = (
    "너는 마케팅 '실무 작업자(Worker)'다. "
    "주어진 캠페인 브리프 맥락에서, 지정된 '하나의 작업'만 구체적으로 수행하라. " # ← 한 작업만!
    "결과는 5줄 이내로 실무에 바로 쓸 수 있게 작성하라."
)
```

"지정된 하나의 작업만"이 핵심입니다. Worker는 전체 캠페인을 보지 않고, 자기에게 주어진 작업 하나에만 집중하죠. 그래서:

- 같은 Worker 함수를 작업마다 재사용 (work(brief, 작업1), work(brief, 작업2)...)
 - 작업이 독립적이라 병렬 실행 가능
 - 한 작업 실패해도 그것만 재시도 (07번)

Worker는 캠페인 브리프(맥락)와 자기 작업 하나를 받아 결과를 냅니다. 맥락을 주는 건 "전체 그림 안에서 이 작업을 하라"는 뜻이죠.

4. 에이전트 간 데이터는 "구조화"로

가장 중요한 설계 포인트입니다. Planner가 작업을 **자연어** 문단으로 주면 Worker에게 넘길 때 깨집니다.

[나쁜 예 — 자유 텍스트]

Planner: "먼저 타깃을 분석하고, 그 다음 카피를 쓰고, 예산도 배분하세요."

→ 코드가 이 문장을 어떻게 작업 리스트로 쪼개지? (파싱 지옥)

[좋은 예 — 구조화 출력]

Planner: ["타깃 분석", "카피 작성", "예산 배분"] (JSON list)

→ for task in subtasks: work(task) (바로 반복 가능!)

그래서 Planner 출력을 **JSON** 리스트로 강제합니다(8강 `with_structured_output`).

```
from pydantic import BaseModel, Field

class Plan(BaseModel):
    """Planner가 만드는 실행 계획(구조화 출력 스키마)."""
    subtasks: list[str] = Field(description="이 캠페인을 위해 필요한 하위 작업 3~4개")
```

`list[str]`로 강제하면, LLM이 무슨 말을 하든 결과가 리스트로 보장됩니다. 코드는 `for task in plan.subtasks`로 바로 돌리죠. 파싱 실수가 사라집니다.

이건 백엔드에서 서비스 간 통신에 **JSON 스키마(DTO)**를 쓰는 것과 똑같은 원칙입니다 — "자유 텍스트 말고 정해진 형식으로 주고받자."

5. 온도(temperature) 분리

Planner와 Worker는 다른 **temperature**를 씁니다.

```
llm_plan = get_chat(temperature=0)    # 계획은 일관성·재현성이 중요 → 0
llm_work = get_chat(temperature=0.5)  # 카피 등 창의 작업은 다양성이 도움 → 0.5
```

```
Planner (temperature=0):
    계획은 매번 비슷해야 함 (재현성) → 무작위성 제거
Worker (temperature=0.5):
    카피 · 아이디어는 창의성 · 다양성이 필요 → 약간 높임
```

같은 LLM이라도 역할에 맞게 온도를 조절합니다. 분류·계획은 0, 창작은 0.5~0.7 — 이 감각이 중요하죠(2강 temperature).

6. 취합 — 산출물 모으기

Worker들의 산출물을 모아 하나의 문서로 정리합니다.

```
report = [f"# 캠페인 실행안\n\n> {brief_text}\n"]
for i, (task, output) in enumerate(outputs, 1):
    report.append(f"## {i}. {task}\n{output}\n")
final = "\n".join(report)
```

각 작업을 **## 1. 타겟 분석**, **## 2. 카피 작성** 처럼 섹션으로 묶어 읽기 좋은 실행안을 만듭니다. Planner가 쪼갬 작업이 그대로 문서의 목차가 되는 셈이죠.

핵심 정리

- **Planner**(전략가): 실행 안 하고 작업을 **JSON 리스트로 분해**("실행 금지" 명시).
- **Worker**(실무자): 받은 **한 작업만** 깊이 수행(재사용·병렬·재시도 유리).
- 에이전트 간 데이터는 **구조화 출력**(list[str])으로 → 파싱 실수 제거(DTO 원칙).
- 온도 분리: Planner 는 **0**(재현성), Worker 는 **0.5**(창의성).
- 취합: Worker 산출물을 **섹션으로 묶어** 최종 실행안.

Supervisor vs Planner/Worker

Supervisor 는 한 질문을 **한 명에게 위임**(수평, 전문가 중 택 1)했습니다. Planner/Worker 는 한 작업을 **여러 단계로 분해**(수직, 계획→실행)합니다. 둘은 목적과 협업 형태가 다르며, 실무에서는 **함께** 쓰이기도 합니다.

1. 항목별 비교

구분	Supervisor	Planner / Worker
목적	한 질문을 한 명에게 위임	한 작업을 여러 단계로 분해
협업 형태	수평(전문가 중 택 1)	수직(계획 → 실행)
산출물	위임된 에이전트의 답	여러 산출물의 취합본
비유	API Gateway	작업 큐 + 워커
에이전트 수	미리 정해진 전문가들	Worker 는 작업 수만큼 반복

2. 수평 분리 vs 수직 분리

핵심 차이는 나누는 방향입니다.

[Supervisor — 수평 분리] "누가 할 일인가?"를 결정

- 질문 → 라우터 → 전문가(추천 / 정책 / 주문) 중 **1명만** 골라 위임

[Planner/Worker — 수직 분리] "어떤 순서로 무엇을 할 것인가?"를 결정

- 작업 → **Planner**가 계획(작업1, 작업2, 작업3) → **Worker**가 작업1 실행 → 작업2 실행 → 작업3 실행

Supervisor는 선택(여럿 중 하나), Planner/Worker는 분해(하나를 여럿으로). 방향이 정반대죠.

3. 언제 무엇을 쓰나

[Supervisor가 맞는 경우]

- 질문 유형이 명확히 같림 (추천 vs 정책 vs 주문)
- 한 질문은 보통 한 전문가가 처리하면 끝
- 예: 고객 문의 라우팅

[Planner/Worker가 맞는 경우]

- 하나의 큰 작업을 여러 단계로 쪼개야 함
- 단계들이 모여 하나의 산출물을 만들
- 예: 캠페인 기획, 리포트 작성, 프로젝트 분해

"이건 누구 일?" 이면 Supervisor, "이걸 어떻게 단계별로?" 면 Planner/Worker 입니다. 질문이나 작업이나, 선택이나 분해냐로 구분하죠.

4. 둘은 함께 쓰일 수 있다

실무에서는 두 패턴이 결합되기도 합니다.

```
[Planner/Worker 안에 Supervisor]
Planner가 작업을 쪼갬 → [타깃분석, 카피작성, 예산배분]
↓ 각 작업마다
Supervisor가 알맞은 전문 Worker에게 위임
- 타깃분석 → 데이터분석 전문 에이전트
- 카피작성 → 카피라이팅 전문 에이전트
- 예산배분 → 재무 전문 에이전트
```

Planner가 만든 각 작업을, Supervisor가 알맞은 전문 **Worker**에게 보내는 거죠. 이번 강에서는 단순화를 위해 같은 **Worker**가 모든 작업을 처리하지만, 작업별로 전문 Worker를 두고 Supervisor로 라우팅하면 더 강력해집니다.

핵심 정리

- **Supervisor**: 수평 분리, 전문가 중 택 1(누구 일?), API Gateway.
- **Planner/Worker**: 수직 분리, 하나를 여러 단계로 분해(어떻게?), 작업 큐+워커.
- 구분: 질문이나 작업이나, 선택이나 분해냐.
- 둘은 결합 가능 — Planner가 쪼갬 작업을 Supervisor가 전문 Worker에 라우팅.
- **Planner/Worker**는 코드(for 루프)로 흐름 제어 → ai team은 **StateGraph**로 정식화.

따라하기 — Planner 작업 분해

목표

캠페인 브리프를 읽어, Planner가 하위 작업 3~4개를 JSON 리스트로 분해하게 만듭니다. 구조화 출력으로 작업 리스트를 안전하게 받습니다.

0. 실습 준비

```
conda activate agentic
pip install -U langchain langchain-google-genai pydantic pandas
python code/common.py          # GOOGLE_API_KEY: True 확인
```

- `pydantic`: 구조화 출력 스키마(8강).
- `pandas`: 캠페인 CSV 읽기(11강).
- 데이터: `data/marketing_brief.csv` (campaign_id, title, category, target, budget, key_offer).

1. 시작부와 데이터 로드

`code/ch26_role_agent.py` 를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch26_role_agent.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat, DATA

import pandas as pd
from pydantic import BaseModel, Field          # 구조화 출력(JSON 스키마) 정의용
from langchain_core.messages import SystemMessage, HumanMessage

# [왜 Planner/Worker 분리] 한 LLM에게 '계획도 세우고 실행도 하라'고 하면 둘 다 어설퍼진다.
# Planner(계획 세우는 역할)가 작업을 잘게 쪼개고, Worker(실행 역할)가 한 작업씩
# 집중 수행하면 각자 잘하는 일에만 몰두해 품질이 올라간다. (25강 역할 분리의 연장선)

def load_brief_text(campaign_id: str = "CMP02") -> str:
    """캠페인 ID로 브리프 1건을 읽어 LLM에 넣을 한 줄 텍스트로 만든다."""
    brief = pd.read_csv(DATA / "marketing_brief.csv")
    campaign = brief[brief["campaign_id"] == campaign_id].iloc[0] # 캠페인 1건 선택
    return (
        f"캠페인명: {campaign.title} / 카테고리: {campaign.category} / "
        f"타겟: {campaign.target} / 예산: {campaign.budget:,}원 / "
        f"핵심혜택: {campaign.key_offer}"
    )
```

`load_brief_text` 는 CSV의 한 행을 LLM이 읽기 좋은 한 줄 텍스트로 만듭니다. 여러 컬럼을 "캠페인명: ... / 카테고리: ..." 형태로 합치죠.

2. 구조화 출력 스키마

Planner의 출력을 **JSON** 리스트로 강제할 스키마입니다(02번 원칙).

```
class Plan(BaseModel):
    """Planner가 만드는 실행 계획(구조화 출력 스키마)."""
    # [왜 구조화 출력] 자유 텍스트가 아니라 list[str] 형태로 강제해, 코드가
    # 결과를 바로 반복(for)할 수 있게 한다. 파싱 실수가 사라진다.
    subtasks: list[str] = Field(description="이 캠페인을 위해 필요한 하위 작업 3~4개")
```

`subtasks: list[str]` — 무슨 일이 있어도 결과가 문자열 리스트로 나옵니다. `Field(description=...)` 은 LLM에게 "이 필드에 뭘 채울지" 알려주는 설명이죠.

3. Planner 함수

역할 프롬프트와 `with_structured_output` 으로 작업을 분해합니다.

```
PLANNER_SYSTEM = (
    # Planner 역할 고정 프롬프트 — '쪼개기만, 실행은 금지'
    "너는 마케팅 캠페인 '기획 설계자(Planner)'다. "
    "주어진 캠페인 브리프를 보고, 실행에 필요한 하위 작업을 3~4개로 쪼개라. "
    "작업은 서로 겹치지 않게, 한 문장 명령형으로 작성하라. 실행은 하지 마라."
)

def plan(llm, brief_text: str) -> list[str]:
    """브리프를 보고 하위 작업 리스트(JSON)로 분해한다."""
    planner = llm.with_structured_output(Plan) # 출력을 Plan 스키마로 강제
    result = planner.invoke([
        SystemMessage(PLANNER_SYSTEM), # 역할 지시
        HumanMessage(f"[캠페인 브리프]\n{brief_text}"), # 입력 데이터
    ])
    return result.subtasks # 하위 작업 문자열 리스트
```

핵심

- `llm.with_structured_output(Plan)` — 이 LLM은 이제 **Plan** 스키마로만 답합니다.
- `SystemMessage` (역할) + `HumanMessage` (데이터) — 역할과 입력을 분리(2강 메시지 구조).
- `result.subtasks` — `.invoke` 결과가 `Plan` 객체라, `.subtasks` 로 리스트를 바로 꺼냄.

4. 실행해 확인

```
if __name__ == "__main__":
    brief_text = load_brief_text("CMP02")
    print("[브리프]", brief_text)
    print()

    llm_plan = get_chat(temperature=0) # 계획은 재현성 → 0
    subtasks = plan(llm_plan, brief_text)
    print("[Planner 분해 결과]")
    for i, t in enumerate(subtasks, 1):
        print(f"    {i}. {t}")
```

예상 출력

[브리프] 캠페인명: 신학기 전자기기 프로모션 / 카테고리: 전자기기 / 타겟: 대학생 / 예산: 8,000,000 원 / 핵심혜택: 이어버드+보조배터리 번들

[Planner 분해 결과]

1. 대학생 타겟의 핵심 니즈와 구매 동기를 분석하라
2. '이어버드+보조배터리 번들' 혜택을 강조한 광고 카피 3 종을 작성하라
3. 800 만원 예산을 채널별로 배분하는 방안을 제안하라
4. 캠페인 성과를 측정할 핵심 지표(KPI)를 정의하라

관찰 포인트

- Planner가 캠페인을 **3~4개** 작업으로 깔끔히 쪼갰습니다. 각 작업은 한 문장 명령형이죠.
- "실행은 하지 마라"를 지켜, 카피 자체는 안 쓰고 "카피를 작성하라"는 작업만 만들었습니다.
- 결과가 리스트라 `for` 로 바로 순회 가능 — 05번 Worker 루프에 그대로 넘어갑니다.

5. 구조화 출력의 가치 재확인

만약 구조화 출력 없이 자유 텍스트로 받았다면:

Planner: "먼저 대학생 타겟을 분석하고, 그 다음 번들 혜택 카피를 쓰고,
예산은 800만원을 채널별로 나누면 됩니다."
→ 이걸 어떻게 ['작업1', '작업2', '작업3']로 쪼개지? (정규식? 줄바꿈? 번호?)
→ 형식이 매번 달라 파싱이 깨짐

with_structured_output(Plan) 덕에 **항상 깔끔한 리스트**를 받습니다. "LLM 이 자유롭게 답하면 코드가 고생한다 → 형식을 강제하자"는 게 핵심이죠. 에이전트 간 데이터 전달의 기본기입니다.

핵심 정리

- load_brief_text(): 캠페인 CSV 한 행을 **LLM 용 한 줄 텍스트**로.
- Plan(subtasks: list[str]): 출력을 **JSON 리스트로 강제**하는 스키마.
- PLANNER_SYSTEM: "쪼개기만, **실행 금지**" + "겹치지 않게, 명령형".
- llm.with_structured_output(Plan) → .subtasks 로 작업 리스트 획득.
- 구조화 출력 덕에 결과가 **항상 리스트** → Worker 루프에 안전 전달.

따라하기 — Worker 작업 실행

목표

Planner가 만든 작업 리스트의 각 작업을 **Worker**가 하나씩 실행하게 만듭니다. Worker는 캠페인 맥락 안에서 지정된 한 작업만 구체적으로 수행합니다.

1. Worker 함수

Worker는 받은 한 작업만 깊이 수행합니다.

```
WORKER_SYSTEM = (          # Worker 역할 고정 프롬프트 — '딱 한 작업만' 집중
    "너는 마케팅 '실무' 작업자(Worker)'다. "
    "주어진 캠페인 브리프 맥락에서, 지정된 '하나의 작업'만 구체적으로 수행하라. "
    "결과는 5줄 이내로 실무에 바로 쓸 수 있게 작성하라."
)

def work(llm, brief_text: str, subtask: str) -> str:
    """하나의 하위 작업만 받아 구체적으로 수행한 결과를 반환한다."""
    result = llm.invoke([
        SystemMessage(WORKER_SYSTEM),
        HumanMessage(f"[캠페인 브리프]\n{brief_text}\n\n[수행할 작업]\n{subtask}"), # 맥락+그 작업 하나
    ])
    return result.content
```

핵심

- `WORKER_SYSTEM` — "지정된 하나의 작업만" + "5줄 이내 실무용". 범위를 좁혀 집중시킴.
- `HumanMessage` 에 맥락(브리프) + 작업(subtask) 을 함께 전달. "전체 그림 안에서 이 작업을 하라"는 뜻.
- `work(llm, brief_text, subtask)` — 작업 하나를 인자로 받으니, 작업마다 재호출 가능.

2. 왜 맥락(브리프)을 함께 주나

Worker에게 작업만 주지 않고 캠페인 브리프(맥락)도 함께 줍니다.

[맥락 없이 작업만]

"광고 카피 3종을 작성하라"

→ Worker: "무슨 상품? 누구 대상? 예산은?" (정보 부족 → 일반적인 카피)

[맥락 + 작업]

"[브리프] 신학기/대학생/이어버드+보조배터리 번들/800만원

[작업] 광고 카피 3종을 작성하라"

→ Worker: 대학생 · 번들에 딱 맞는 구체적 카피

같은 작업이라도 맥락이 있어야 구체적인 결과가 나옵니다. Worker는 작업 하나에 집중하되, 전체 캠페인 맥락 안에서 일하는 거죠.

3. Worker는 같은 함수를 반복 호출

Planner가 만든 작업 리스트를 하나씩 순회하며 Worker를 호출합니다.

```
llm_work = get_chat(temperature=0.5)    # 카피 등 창의 작업은 다양성이 도움 → 0.5

outputs = []
for i, subtask in enumerate(subtasks, 1):
    print(f"  - 작업 {i} 수행: {subtask}")
    result = work(llm_work, brief_text, subtask)  # 같은 work() 함수를 작업마다 재호출
    outputs.append((subtask, result))            # (작업, 산출물) 튜플로 보관
```

같은 `work()` 함수를 작업마다 호출합니다. 작업 1, 작업 2, 작업 3... 각각 독립적으로 실행되죠. 이 독립성 덕에:

- 재사용: 함수 하나로 모든 작업 처리
- 병렬화: 작업끼리 독립이라 동시 실행 가능 (`concurrent.futures`)
- 재시도: 한 작업 실패해도 그것만 다시 (07번)

(작업, 산출물) 튜플로 묶어 보관해, 나중에 취합할 때 "어떤 작업의 결과인지" 알 수 있게 합니다.

4. 실행해 확인

```
if __name__ == "__main__":
    brief_text = load_brief_text("CMP02")
    llm_plan = get_chat(temperature=0)
    llm_work = get_chat(temperature=0.5)

    subtasks = plan(llm_plan, brief_text)          # 04번 Planner
    print("[작업 분해 완료]", len(subtasks), "개\n")

    for i, subtask in enumerate(subtasks, 1):
        print(f"[작업 {i}] {subtask}")
        result = work(llm_work, brief_text, subtask) # Worker 실행
        print(result)
        print("-" * 50)
```

예상 출력

[작업 분해 완료] 3 개

[작업 1] 대학생 타킷의 핵심 니즈와 구매 동기를 분석하라

- 가성비 증시: 한정된 예산으로 최대 효용 추구
- 휴대성: 강의실 · 카페 · 이동 중 사용
- 트렌드 민감: 또래 · SNS 영향 큼

[작업 2] '이어버드+보조배터리 번들' 혜택을 강조한 광고 카피 3종을 작성하라

- 1) "강의실에서 카페까지, 하루 종일 끊김 없이"
- 2) "신학기엔 풀충전 — 이어버드 + 보조배터리 한 세트"
- 3) "방전 걱정 끝, 공부도 음악도 멈추지 않게"

[작업 3] 800만원 예산을 채널별로 배분하는 방안을 제안하라

- 인스타/틱톡 광고: 400만원 (타킷 도달)
- 대학 커뮤니티 제휴: 250만원
- 인플루언서 협업: 150만원

관찰 포인트

- 각 Worker가 자기 작업에만 집중해 구체적인 결과를 냈습니다.
- 작업 2(카피)는 대학생·번들에 딱 맞는 문구 — 맥락을 반영했죠.
- 작업 3(예산)은 800만원을 실제로 배분 — 구체적 숫자까지.

각 작업이 독립적으로 잘 처리됐습니다. 이제 06번에서 이 산출물들을 하나의 실행안으로 취합합니다.

5. 병렬화 가능성 (미리보기)

Worker 작업이 독립적이라, 사실 동시에 실행할 수도 있습니다.

```
# 참고: 작업이 독립적이라 병렬 실행도 가능 (선택적 최적화)
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=3) as ex:
    results = list(ex.map(lambda t: work(llm_work, brief_text, t), subtasks))
outputs = list(zip(subtasks, results))
```

작업 3개를 순차로 하면 3배 시간이 걸리지만, 병렬로 하면 거의 1개 시간에 끝나죠. 단, 이번 강은 이해를 위해 순차 실행으로 두고, 병렬·재시도 같은 견고화는 07번과 29강에서 다룹니다. "독립적이면 병렬 가능"이라는 점만 기억하세요.

핵심 정리

- `work()`: Worker 가 받은 한 작업만 구체적으로 수행(5 줄 이내 실무용).
- 맥락(브리프) + 작업을 함께 줘야 구체적 결과가 나옴.
- 같은 `work()` 함수를 작업마다 재호출(for 루프) → 재사용·독립.
- (작업, 산출물) 튜플로 보관 → 취합 시 추적 가능.
- 작업이 독립적이라 병렬화·재시도에 유리.

따라하기 — 오케스트레이터 취합

목표

Planner(계획)와 Worker(실행)를 하나의 오케스트레이터 함수로 묶고, Worker 산출물을 모아 하나의 캠페인 실행안으로 취합합니다.

1. 오케스트레이터 — 계획 → 실행 → 취합

`run_campaign` 이 전체 흐름을 지휘합니다.

```
def run_campaign(brief_text: str) -> str:
    """Planner로 쪼개고, Worker로 하나씩 실행해, 결과를 하나의 실행안으로 취합한다."""
    llm_plan = get_chat(temperature=0)      # [왜 0] 계획은 일관성 · 재현성이 중요
    llm_work = get_chat(temperature=0.5)     # [왜 0.5] 카피 등 창의 작업은 다양성이 도움

    print("[1] Planner 작업 분해 중...")
    subtasks = plan(llm_plan, brief_text)    # 계획 단계
    for i, t in enumerate(subtasks, 1):
        print(f"  서브태스크 {i}: {t}")

    print("\n[2] Worker 실행 중...")
    outputs = []
    for i, t in enumerate(subtasks, 1):      # 하위 작업을 하나씩 순회 실행
        print(f"  - 작업 {i} 수행: {t}")
        outputs.append((t, work(llm_work, brief_text, t)))  # (작업, 산출물) 보관

    print("\n[3] 결과 취합")
    report = [f"# 캠페인 실행안\n\n> {brief_text}\n"]
    for i, (t, out) in enumerate(outputs, 1):
        report.append(f"## {i}. {t}\n{out}\n")
    return "\n".join(report)
```

세 단계가 한 함수에 흐름니다.

```
[1] plan()으로 작업 분해      (Planner, temperature=0)
[2] work()를 작업마다 반복    (Worker, temperature=0.5)
[3] 산출물을 마크다운으로 취합
```

2. "오케스트레이터"란

`run_campaign` 같은 함수를 오케스트레이터(**orchestrator**, 지휘자) 라 부릅니다.

오케스트라 지휘자: 각 연주자(Planner · Worker)에게 언제 무엇을 할지 지시
오케스트레이터 함수: 각 에이전트를 언제 호출할지, 결과를 어떻게 넘길지 제어

오케스트레이터의 책임:

- 순서 제어: Planner 먼저, 그 다음 Worker
- 데이터 전달: Planner 결과(작업 리스트)를 Worker에 넘김
- 취합: Worker 산출물을 모아 최종 결과로

에이전트들은 각자 일만 하고, 흐름은 오케스트레이터가 코드로 제어합니다.

Role base agent 장에서는 for 루프로, ai team 장에서는 StateGraph 로 이 지휘를 합니다.

3. 취합 — 섹션으로 묶기

취합 부분을 자세히 보면:

```
report = [f"# 캠페인 실행안\n\n> {brief_text}\n"] # 제목 + 브리프 요약
for i, (t, out) in enumerate(outputs, 1):
    report.append(f"## {i}. {t}\n{out}\n") # 각 작업을 섹션으로
final = "\n".join(report)
```

```
# 캠페인 실행안
> 캠페인명: 신학기 전자기기 프로모션 / ...

## 1. 대학생 타겟의 핵심 니즈와 구매 동기를 분석하라
(작업1 산출물)

## 2. '번들' 혜택을 강조한 광고 카피 3종을 작성하라
(작업2 산출물)

## 3. 800만원 예산을 채널별로 배분하라
(작업3 산출물)
```

Planner가 쪼갬 작업이 그대로 문서의 목차(## 1., ## 2.)가 됩니다. 계획이 곧 구조가 되는 거죠. 읽는 사람은 "어떤 작업으로 나뉘 어떤 결과가 나왔는지" 한눈에 봅니다.

4. 전체 실행

```
def main():
    brief_text = load_brief_text("CMP02") # 캠페인 1건 텍스트 로드
    print(run_campaign(brief_text))
```

```
if __name__ == "__main__":  
    main()
```

예상 출력

```
[1] Planner 작업 분해 중...  
  서브태스크 1: 대학생 타킷의 핵심 니즈와 구매 동기를 분석하라  
  서브태스크 2: '이어버드+보조배터리 번들' 혜택을 강조한 광고 카피 3종을 작성하라  
  서브태스크 3: 800만원 예산을 채널별로 배분하는 방안을 제안하라  
  
[2] Worker 실행 중...  
  - 작업 1 수행: 대학생 타킷의 ...  
  - 작업 2 수행: '이어버드+보조배터리 번들' ...  
  - 작업 3 수행: 800만원 예산을 ...  
  
[3] 결과 취합  
# 캠페인 실행안  
  
> 캠페인명: 신학기 전자기기 프로모션 / 카테고리: 전자기기 / 타킷: 대학생 / 예산: 8,000,000원 ...  
  
## 1. 대학생 타킷의 핵심 니즈와 구매 동기를 분석하라  
- 가성비와 휴대성을 중시 ...  
  
## 2. '이어버드+보조배터리 번들' 혜택을 강조한 광고 카피 3종 ...  
1) "강의실에서 카페까지, 하루 종일 끊김 없이 — 이어버드+보조배터리 번들" ...  
  
## 3. 800만원 예산을 채널별로 배분하는 방안을 제안하라  
- 인스타/틱톡 광고: 400만원 ...
```

캠페인 브리프 한 줄에서 완성된 실행안이 나왔습니다. Planner가 쪼개고, Worker가 채우고, 오케스트레이터가 엮은 결과죠.

5. 단일 호출 vs Planner/Worker 비교

같은 캠페인을 한 번의 LLM 호출로 처리했다면 어땠을까요?

[단일 호출] "캠페인 다 준비해줘"
→ 타킷 분석은 한 줄, 카피는 대충, 예산은 깜빡... (어설피)

[Planner/Worker]
→ 각 작업을 독립적으로 깊이 처리 → 빠짐없이 균일한 품질

작업을 나눠 각자 집중하니 각 부분의 품질이 고르고 깊습니다. "한 번에 다"보다 "나눠서 차근차근"이 나은 거죠. 이게 Planner/Worker의 가치입니다.

6. 이 구조의 확장성

`run_campaign`은 작은 함수지만, 확장 여지가 많습니다.

- Worker 병렬화: for 루프 → ThreadPoolExecutor (05번)
- Worker 재시도: 실패 시 다시 (07번 신규)
- Reviewer 추가: 취합 후 검토 단계 (09번 실습)
- 전문 Worker: 작업별 다른 에이전트 (03번 + 25강 Supervisor)
- StateGraph화: for 루프 → 그래프 (27강)

지금은 가장 단순한 순차 버전입니다. 다음 절에서 실패 처리·재시도를 더해 견고하게 만들고, ai team 장에서 StateGraph 로 발전시킵니다.

핵심 정리

- `run_campaign()`: 계획 → 실행 → 취합을 묶은 오케스트레이터.
- 오케스트레이터 = 흐름 지휘자(순서·데이터 전달·취합을 코드로 제어).
- 취합: 작업을 섹션(## 1.)으로 묶어 → Planner 계획이 곧 문서 목차.
- 단일 호출보다 품질이 고르고 깊음(각 작업에 집중).
- 확장 여지: 병렬화·재시도·Reviewer·전문 Worker·StateGraph 화.

Worker 실패 처리와 재시도

`run_campaign`은 Worker 가 한 번이라도 예외를 던지면 전체가 멈춥니다.

실무에선 일시적 오류(타임아웃 등)가 흔하죠. Worker 실패 시 최대 2 회 재시도하고, 그래도 실패하면 스킵·기록하고 끝까지 진행하는 견고한 오케스트레이터를 만듭니다.

1. 한 작업의 실패가 전체를 멈춘다

06번 코드의 약점을 봅시다.

```
for i, t in enumerate(subtasks, 1):
    outputs.append((t, work(llm_work, brief_text, t))) # work()가 예외 던지면?
```

`work()`가 작업 2에서 예외를 던지면(타임아웃·네트워크·rate limit), 전체 **for** 루프가 죽습니다.

작업 1 성공 → 작업 2 실패(예외!) → ✖ 전체 중단
→ 작업 1의 산출물도 못 받고, 작업 3은 시작도 못 함

4개 작업 중 1개가 일시적으로 실패했다고 나머지 3개까지 버리는 건 아깝습니다. 실무에선 일시적 오류가 흔하니, 부분 실패를 허용해야 하죠.

2. 견고화 전략 — 재시도 → 스킵 → 계속

Worker 실패 시:

1. 재시도 (일시적 오류 흡수) — 최대 2회
2. 그래도 실패 → 스킵 + 기록
3. 다음 작업으로 계속 진행 (전체는 안 멈춤)

핵심 철학: 한 작업의 실패가 전체를 멈추면 안 된다(부분 실패 허용). 23강 재귀 수정의 재시도, 24강 예외 처리의 정신을 Worker에 적용합니다.

```
from ch26_role_agent import load_brief_text, plan, work # 계획·실행 로직 재사용
```

```
MAX_RETRY = 2 # 최초 1회 + 재시도 2회 = 최대 3번 시도
```

추가 설치 라이브러리는 없습니다(표준 try/except + 기존 함수 재사용).

3. 재시도 래퍼 — work_with_retry

`work()`를 감싸 최대 `MAX_RETRY` 회 재시도하는 래퍼입니다.

```
def work_with_retry(llm, brief_text: str, subtask: str, idx: int, total: int):
    """work()를 감싸 최대 MAX_RETRY 회 재시도. (성공여부, 결과 or 오류메시지) 반환."""
    for attempt in range(1, MAX_RETRY + 2): # 1, 2, 3 (최초 + 재시도 2회)
        try:
            print(f"작업 {idx}/{total} 수행 중 (시도 {attempt})...")
            out = work(llm, brief_text, subtask)
            return True, out # 성공 → (True, 산출물)
```

```

except Exception as e:                # 타임아웃 · 네트워크 · 파싱 등 모든 실패 흡수
    print(f"    [실패] {type(e).__name__}: {e}")
    if attempt <= MAX_RETRY:
        print(f"        → 재시도합니다 ({attempt}/{MAX_RETRY})")
    else:
        print(f"        → 최대 재시도 초과. 이 작업은 스킵합니다.")
        return False, f"{type(e).__name__}: {e}" # 실패 → (False, 오류메시지)

```

설계 포인트

- `range(1, MAX_RETRY + 2)` → `1, 2, 3`: 최초 1회 + 재시도 2회 = 최대 3번 시도.
- 성공하면 즉시 `(True, 산출물)` 반환(더 재시도 안 함).
- 모든 시도 실패하면 `(False, 오류메시지)` 반환 → 호출부가 스킵 처리.
- `(성공여부, 결과)` 튜플 패턴 — 24강 `load_facts_safe`의 `ok` 플래그와 같은 정신.

4. 견고한 오케스트레이터

`run_campaign` 을 재시도·스킵을 지원하게 개선합니다.

```

def run_campaign_robust(brief_text: str, fail_indices=None) -> str:
    """계획 → (재시도 포함) 실행 → 취합. 실패 작업은 스킵하고 끝까지 진행한다."""
    fail_indices = set(fail_indices or []) # 시연용: 일부러 실패시킬 작업 번호
    llm_plan = get_chat(temperature=0)
    llm_work = get_chat(temperature=0.5)

    subtasks = plan(llm_plan, brief_text)
    total = len(subtasks)
    outputs = [] # (작업, 산출물) — 성공한 것만
    failed = [] # (번호, 작업, 오류) — 스킵한 것

    for i, t in enumerate(subtasks, 1):
        ok, info = work_with_retry(llm_work, brief_text, t, i, total)
        if ok:
            outputs.append((t, info))
            print(f"    작업 {i}/{total} 완료")
        else:
            failed.append((i, t, info)) # 스킵하고 다음 작업으로 진행

    # 취합 — 성공분 + 미완료 목록
    print(f"\n    성공 {len(outputs)}개 / 실패(스킵) {len(failed)}개 / 전체 {total}개")
    report = [f"# 캠페인 실행안\n\n> {brief_text}\n"]
    for i, (t, out) in enumerate(outputs, 1):
        report.append(f"## {i}. {t}\n{out}\n")

```

```

if failed:
    report.append("## ⚠️ 미완료(재시도 후 스킵된) 작업")
    for i, t, err in failed:
        report.append(f"- {t} - {err}")
    return "\n".join(report)

```

핵심

- `outputs` (성공)와 `failed` (스킵)를 따로 모음.
- 실패해도 `continue` 처럼 다음 작업으로 진행 → 전체는 안 멈춤.
- 취합 시 성공분은 정상 섹션으로, 미완료 작업은 ⚠️ 목록으로 명시(투명성).

5. 실행 — 일부러 실패시켜 시연

작업 2를 일부러 매번 실패시켜 재시도·스킵 흐름을 보여줍니다.

```

if __name__ == "__main__":
    brief_text = load_brief_text("CMP02")
    print("(2번 작업은 일부러 매번 실패시켜 재시도·스킵을 보여줍니다)")
    report = run_campaign_robust(brief_text, fail_indices=[2])
    print(report)

```

예상 출력

(2번 작업은 일부러 매번 실패시켜 재시도·스킵을 보여줍니다)

```

작업 1/3 수행 중 (시도 1)...
작업 1/3 완료
작업 2/3 수행 중 (시도 1)...
  [실패] TimeoutError: 모의 일시 오류
  → 재시도합니다 (1/2)
작업 2/3 수행 중 (시도 2)...
  [실패] TimeoutError: 모의 일시 오류
  → 재시도합니다 (2/2)
작업 2/3 수행 중 (시도 3)...
  [실패] TimeoutError: 모의 일시 오류
  → 최대 재시도 초과. 이 작업은 스킵합니다.
작업 3/3 수행 중 (시도 1)...
작업 3/3 완료

성공 2개 / 실패(스킵) 1개 / 전체 3개

```

```
# 캠페인 실행안
...
## 1. 대학생 타겟 분석
...
## 2. 800만원 예산 배분
...
## ⚠️ 미완료(재시도 후 스킵된) 작업
- 광고 카피 3종을 작성하라 — TimeoutError: 모의 일시 오류
```

관찰 포인트

- 작업 2가 3번 시도(최초+재시도 2회) 후 스킵됐지만, **작업 1·3은 정상 완료**됐습니다.
- 한 작업이 죽어도 전체는 **살아남아** 부분 산출물(2개)을 확보했죠.
- 실패한 작업은 ⚠️ 목록에 **투명하게 기록** — 나중에 사람이 그것만 다시 처리.

6. 왜 부분 실패 허용이 중요한가

[전부 아니면 전무 (all-or-nothing)]

4개 작업 중 1개 실패 → 0개 산출물 (다 버림)

[부분 실패 허용 (graceful degradation)]

4개 작업 중 1개 실패 → 3개 산출물 확보 + 1개는 추적

실무에선 **"완벽한 0개보다 불완전한 3개"**가 낫습니다. 마케팅 실행안의 4개 섹션 중 3개라도 있으면 일을 시작할 수 있고, 빠진 1개만 따로 처리하면 되죠. 24강 폴백("불완전해도 결과"), 25강 견고성과 같은 정신입니다.

Worker가 독립적이기에 이게 가능합니다(05번). 작업끼리 의존했다면 하나 실패가 연쇄로 무너졌겠죠. 독립성이 견고성의 기반입니다.

핵심 정리

- Worker 한 번 실패에 **전체가 멈춤** → 일시적 오류에 취약.
- 전략: **재시도(일시 오류 흡수) → 스킵·기록 → 끝까지 진행**.
- `work_with_retry`: 최대 MAX_RETRY 회 재시도, (성공여부, 결과) 반환.
- 견고한 오케스트레이터: 성공/실패를 따로 모아 **부분 산출물 확보** + ⚠️ 기록.
- **"완벽한 0개보다 불완전한 3개"**(graceful degradation).
- Worker 독립성이 부분 실패 허용의 기반.

2. AI team

if 문 대신 그래프로 흐름을 선언한다

role based agent 장에서는 파이썬 **for** 루프로 에이전트들의 흐름을 직접 제어했습니다.

단계가 2~3 개면 괜찮지만, "결과가 나쁘면 되돌아가 재시도", "조건에 따라 단계 건너뛰기" 같은 **갈림길**이 생기면 if-while 이 얹혀 금세 엉망이 됩니다.

LangGraph 의 **StateGraph** 는 흐름을 **노드(작업)**와 **엣지(연결)**로 선언해, 복잡한 협업도 한눈에 보이게 만듭니다.

1. 1장의 방식 — 코드로 흐름을 "제어"

Role based agent 장에서 오케스트레이터는 이렇게 흐름을 직접 짭니다.

```
# 26강 — for 루프로 흐름 제어
subtasks = plan(llm_plan, brief_text)      # 1단계
outputs = []
for t in subtasks:                          # 2단계
    outputs.append((t, work(llm_work, brief_text, t)))
report = take(outputs)                     # 3단계
```

흐름이 일직선(계획 → 실행 → 취합)이라 **for** 루프로 충분했습니다. 깔끔하죠.

2. 흐름에 "갈림길"이 생기면?

그런데 실무 요구가 이렇게 늘어난다고 해봅시다.

- "분석이 부실하면 → 다시 조사하게 되돌려라" (되돌아가기)
- "리포트 품질이 미달이면 → Writer를 다시 돌려라" (반복)
- "급한 건이면 → 분석 단계를 건너뛰어라" (건너뛰기)

이걸 **for** / **if** / **while** 로 짜기 시작하면:

```
# 갈림길이 늘어난 코드 (점점 엉킴)
while True:
    analysis = analyst(raw)
    if is_good(analysis):
        break
    elif retries < 2:
        raw = research_again(); retries += 1
    else:
        break
if urgent:
    report = writer(raw)      # 분석 건너뛰기
else:
    report = writer(analysis)
# ... 조건이 더 늘면? 분기가 코드 곳곳에 흩어져 스파게티가 된다
```

흐름이 코드 곳곳에 흩어져 전체 그림이 안 보입니다. "지금 어디서 어디로 가지?"를 코드를 다 읽어야 알 수 있죠. 단계를 하나 추가하려면 여러 군데를 고쳐야 합니다.

3. StateGraph — 흐름을 "선언"

`StateGraph` 는 발상을 바꿉니다. "어떻게 제어할지"를 코드로 짜는 대신, "무엇이 무엇으로 이어지는지"를 그래프로 선언합니다.

```
# 흐름을 한 곳에 그래프로 선언
g.add_edge(START, "researcher")      # 시작 → 리서처
g.add_edge("researcher", "analyst")  # 리서처 → 분석가
g.add_edge("analyst", "writer")      # 분석가 → 작성자
g.add_edge("writer", END)            # 작성자 → 끝
```

START → researcher → analyst → writer → END

흐름이 한 곳에 그림처럼 모여 있습니다. 누가 봐도 "리서처 다음 분석가, 그다음 작성자"임이 보이죠. 갈림길이 필요하면 `add_conditional_edges` 한 줄로 "검수 → (부실)되돌리기 / (충분)진행" 을 선언합니다(06번에서 실습).

4. 비유 — 순서도를 그대로 코드로

[for/if 방식] "어떻게 갈지" 매 갈림길마다 코드로 명령
→ 흐름이 코드에 녹아 안 보임

[StateGraph] "어디서 어디로" 지도를 먼저 그림
→ 노드(역)와 엣지(선로)가 그대로 보임

백엔드 개발자라면 **상태 기계(State Machine)** 나 **워크플로우 엔진(DAG)** 과 같다고 보면 됩니다. 흐름(라우팅)을 선언적으로 표현하는 것이죠.

5. 이번 강에서 만들 것 — AI 팀

27강에서는 진짜 "AI 팀"을 만듭니다. 세 명의 전문가가 **공유 문서(State)** 를 차례로 채워 갑니다.

Researcher(사실 정리) → Analyst(강약점 분석) → Writer(보고서 작성)

승승장구물의 경쟁사 데이터(`competitor_data.csv`) 를 입력으로, 경쟁사 분석 리포트(마크다운) 를 자동 생성하는 팀입니다. 먼저 가장 단순한 일직선 파이프라인으로 StateGraph 사용법을 익히고(04-05번), 그다음 조건부 분기(06번)를 더해 "왜 그래프인가"를 체감합니다.

📌 솔직한 안내 — 이번 일직선 예제는 사실 **26강 for** 루프로도 충분합니다. StateGraph의 진가는 분기·되돌아가기·반복이 생길 때 나옵니다. 그래서 이 강은 "단순한 형태로 그래프 문법을 익히고 → 분기를 더해 진가를 보는" 순서로 설계했습니다. 흐름이 일직선이면 **for**, 갈림길이 생기면 **StateGraph** — 이 한 줄을 기억하세요.

핵심 정리

- 1 장은 **for/if** 로 흐름을 제어 — 일직선엔 충분하지만 갈림길이 늘면 스파게티가 됨.
- StateGraph 는 흐름을 **노드·엣지**로 선언 — 전체 그림이 한 곳에 보이고 유지보수가 쉬움.
- 이번 강은 **Researcher → Analyst → Writer** AI 팀을 그래프로 만듦.
- 흐름이 일직선이면 **for**, 갈림길이 생기면 **StateGraph**.

Researcher → Analyst → Writer 파이프라인

AI 팀은 역할이 다른 세 에이전트가 공유 문서(State)를 차례로 채워 가는 파이프라인입니다. **Researcher**가 사실을 정리하면, **Analyst**가 그것으로 분석하고, **Writer**가 보고서로 다듬습니다. 각 역할은 앞 단계 산출물만 입력으로 받습니다.

1. 세 역할

승승장구물 경쟁사 분석을 예로, 세 명이 분업합니다.

`competitor_data.csv` (원천 데이터) → **Researcher**(사실만 정리, 해석 금지) → **Analyst**(강점·약점·기회 분석) → **Writer**(보고서로 변환) → 마크다운 리포트(최종)

- **Researcher**(리서처) — 원천 데이터를 읽어 사실만 항목별로 정리합니다. 해석은 하지 않습니다.
- **Analyst**(분석가) — 정리된 사실로 경쟁사별 강점·약점과 승승장구물의 기회를 분석합니다.
- **Writer**(작성자) — 분석 결과를 임원 보고용 마크다운 리포트(제목·요약·정리·결론)로 변환합니다.

2. 왜 역할을 나누나

"경쟁사 분석 리포트 만들어줘"를 한 번의 LLM 호출로 시키면 어떨까요?

[한 번에 다]

- 사실 · 분석 · 서술이 뒤섞여, 근거가 약하거나 형식이 들쭉날쭉
- "분석이 부실한데 글만 그럴듯한" 보고서가 나오기 쉬움

[역할 분리]

- Researcher가 사실을 탄탄히 → Analyst가 그 위에서 깊이 분석
- Writer는 잘 정리된 분석만 글로 → 각 단계 품질이 안정적

각 역할이 자기 일에만 집중하니 품질이 고릅니다.

Agent workflow 강(전문가 팀), role based agent 강(Planner/Worker)에서 배운 "나눠서 각자 잘하는 일에 집중" 원칙을 파이프라인으로 적용한 것입니다.

3. 핵심 — "앞 단계 산출물만" 입력으로

이 파이프라인의 중요한 성질은 각 역할이 앞 단계의 결과만 본다는 것입니다.

Researcher:	원천 데이터	→ 정리된 사실
Analyst:	정리된 사실	→ 분석 (원천 데이터는 안 봄)
Writer:	분석	→ 리포트 (정리된 사실 · 원천은 안 봄)

이렇게 단계가 분리(느슨한 결합) 되어 있어 좋은 점:

- 품질 안정 — 각 단계 입력이 명확해, 앞 단계가 잘하면 뒤 단계도 잘합니다.
- 교체 용이 — Analyst만 더 똑똑한 모델로 바뀌도, 인터페이스(사실 입력 → 분석 출력)가 같으면 됩니다.
- 디버깅 용이 — 결과가 이상하면 어느 단계에서 틀렸는지 추적하기 쉽습니다.

4. 파이프라인 vs 오케스트레이터

26강과 비슷하지만 결정적 차이가 있습니다.

[26강 Planner/Worker]

Planner가 작업을 '여러 개로 쪼개고' → Worker가 각각 '독립' 수행 → 취합
(작업끼리 서로 무관, 병렬 가능)

[27강 파이프라인]

Researcher → Analyst → Writer 가 '한 줄로 이어짐'
(앞 결과가 뒤 입력 = 순서가 중요, 직렬)

Role based agent 강 Worker 들은 서로 독립이라 순서가 상관없었지만(병렬 가능),

ai team 강 세 역할은 앞 결과를 뒤가 받으므로 순서가 핵심입니다.

이런 직렬 의존 흐름을 깔끔히 표현하는 도구가 바로 다음 절의 StateGraph 입니다.

5. 도구를 안 쓰는 이유

이번 세 역할은 외부 도구(검색·DB조회)를 쓰지 않고 입력 텍스트를 가공만 합니다. 그래서

`create_agent` (도구 쓰는 에이전트) 대신, 노드 안에서 `llm.invoke(...)` 만 하면 됩니다.

- 도구가 필요 없음 → 단순 LLM 호출로 충분
- 핵심은 '흐름을 StateGraph로 선언'하는 것
- (도구가 필요하면 노드 안에서 `create_agent`를 불러도 됨)

이번 강의 초점은

에이전트의 **똑똑함**이 아니라 **여러 에이전트를 어떻게 잇는가(오케스트레이션)**입니다.

핵심 정리

- AI 팀 = **Researcher(사실)** → **Analyst(분석)** → **Writer(보고서)** 직렬 파이프라인.
- 각 역할은 **앞 단계 산출물만** 입력으로 받음 → 느슨한 결합·품질 안정·교체 용이.
- 1 장 Worker 는 독립(병렬 가능), 2 장 세 역할은 **직렬 의존**(순서가 핵심).
- 도구를 안 쓰므로 노드는 **단순 llm.invoke** — 초점은 "어떻게 잇는가".

공유 상태(State)와 그래프 구성요소

StateGraph 의 핵심은 **공유 상태(State)** 입니다. 모든 노드가 **하나의 서류철(State)** 을 돌려 보며 자기 칸을 채웁니다. State 는 TypedDict 로 "어떤 칸이 있는지"를 미리 정의하고, 노드는 **바꿀 칸만 dict 로 반환**하면 LangGraph 가 알아서 병합합니다.

1. 공유 상태 = 팀의 서류철

세 역할이 데이터를 주고받는 방법은 **하나의 공유 문서(State)** 입니다.

```
from typing import TypedDict

class TeamState(TypedDict):
    """팀이 공유하는 상태. 각 노드가 자기 칸을 채워 다음 노드로 넘긴다."""
    raw: str          # Researcher 산출물(정리된 사실)
    analysis: str      # Analyst 산출물(강약점 분석)
    report: str        # Writer 산출물(최종 리포트)
```

`TeamState` (서류철)는 세 칸으로 구성되고, 각 노드가 자기 칸을 채웁니다.

칸	내용	채우는 노드
<code>raw</code>	"..."	Researcher
<code>analysis</code>	"..."	Analyst
<code>report</code>	"..."	Writer

각 노드는 이 서류철을 받아 **자기 칸만** 채우고 다음 노드로 넘깁니다.

2. 왜 TypedDict인가

`TypedDict`는 딕셔너리의 "모양(어떤 키에 어떤 타입)"을 미리 정의합니다.

```
# TypedDict 없이 그냥 dict 를 쓰면?
state = {"raw": "...", "analysis": "..."} # 오타(analysis)를 못 잡음 → 런타임에 터짐

# TypedDict 로 정의하면 — 키 · 타입이 '계약'으로 고정
class TeamState(TypedDict):
    raw: str
    analysis: str # 편집기가 오타 · 타입을 잡아줌
    report: str
```

노드 간에 주고받는 데이터의 **계약(contract)**을 명확히 해, "어떤 칸이 오가는지"를 코드로 못박습니다. 8강의 구조화 출력(Pydantic)과 같은 정신 — 형식을 강제해 실수를 줄인다 입니다.

백엔드 개발자에게 State는 요청 컨텍스트나 **DTO(Data Transfer Object)**와 같습니다.

3. 노드는 "바꿀 칸만" 반환한다

가장 중요한 규칙입니다. 노드 함수는 **state** 전체를 다시 만들지 않고, 자기가 바꿀 칸만 **dict**로 반환합니다.

```
def analyst(state: TeamState) -> dict:
    out = llm.invoke([...]).content
    return {"analysis": out} # analysis 칸만 반환 → LangGraph가 기존 state에 병합
```

```
들어온 state: {raw: "사실...", analysis: "", report: ""}
analyst 반환: {analysis: "분석..."} ← 이 칸만
병합된 state: {raw: "사실...", analysis: "분석...", report: ""}
```

`raw` 나 `report`를 건드리지 않아도 됩니다. 반환한 칸만 갱신되고 나머지는 그대로 유지되죠. 덕분에 노드는 자기 일에만 집중합니다.

4. 그래프의 4가지 구성요소

StateGraph는 네 가지로 이뤄집니다.

구성요소	역할	비유(백엔드)
State (TypedDict)	단계 간 주고받는 데이터 묶음	요청 컨텍스트 / DTO
노드(node)	state를 받아 일부를 갱신하는 함수	핸들러 / 미들웨어
엣지(edge)	다음에 실행할 노드 지정	라우팅 규칙
START / END	그래프의 시작 / 끝 지점	진입점 / 종료

START →(edge) [node:researcher] →(edge) [node:analyst] →(edge) [node:writer] →(edge) END

각 노드는 실행되며 state의 한 칸을 갱신합니다 — researcher는 `raw`, analyst는 `analysis`, writer는 `report`.

- 노드 = 일하는 사람(에이전트 함수).
- 엣지 = "이 노드 다음엔 저 노드" 라는 연결선.
- **START/END** = 어디서 시작해 어디서 끝나는지.

이 네 가지만 조합하면, 일직선 파이프라인부터 분기·루프가 있는 복잡한 워크플로우까지 모두 선언할 수 있습니다.

5. 전체 그림 미리보기

다음 절들에서 만들 코드의 뼈대입니다.

```
g = StateGraph(TeamState)          # ① State 모양으로 그래프 시작
g.add_node("researcher", researcher) # ② 노드 등록
g.add_node("analyst", analyst)
g.add_node("writer", writer)
g.add_edge(START, "researcher")    # ③ 엣지로 흐름 연결
g.add_edge("researcher", "analyst")
g.add_edge("analyst", "writer")
g.add_edge("writer", END)
team = g.compile()                  # ④ 실행 가능한 그래프로 컴파일
```

네 단계: **State 정의** → **노드 등록** → **엣지 연결** → **컴파일**. 먼저 노드(역할 함수)를, 그 다음에 엣지 연결과 실행을 다룹니다.

핵심 정리

- 노드들은 하나의 공유 **State(서류철)** 를 돌려 보며 자기 칸을 채운다.
- **TypedDict** 로 State 의 키·타입을 미리 정의 → 노드 간 데이터 계약을 명확히.
- 노드는 **바꿀 칸만 dict 로 반환** → LangGraph 가 기존 State 에 병합(전체 재생성 불필요).
- 그래프 4 요소: **State · 노드 · 엣지 · START/END**.

따라하기 — 3 개 노드(에이전트) 정의

목표

공유 상태 `TeamState` 와 입력 로더를 만들고, **Researcher·Analyst·Writer** 세 노드(에이전트 함수)를 정의합니다. 실제 파일은 `code/ch27_ai_team.py` 입니다.

0. 실습 준비

```
conda activate agentic

# 27장에서 쓰는 라이브러리
# - langgraph          : StateGraph (노드 · 엣지로 워크플로우 선언)
# - langchain-google-genai : Gemini 챗 모델 (get_chat 내부에서 사용)
pip install -U langgraph langchain-google-genai

python code/common.py      # GOOGLE_API_KEY: True 확인
```

- 데이터: `data/competitor_data.csv` (컬럼: `company, main_category, rating, strength, weakness`).

1. 시작부 & 공유 상태

`code/ch27_ai_team.py` 를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch27_ai_team.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import get_chat, DATA

import pandas as pd
from typing import TypedDict
from langchain_core.messages import SystemMessage, HumanMessage
# StateGraph: 노드(에이전트)들을 엮기로 이어 정해진 순서로 흐르게 하는 상태머신.
from langgraph.graph import StateGraph, START, END

class TeamState(TypedDict):
    """팀이 공유하는 상태. 각 노드가 자기 칸을 채워 다음 노드로 넘긴다."""
    raw: str          # Researcher 산출물(정리된 사실)
    analysis: str      # Analyst 산출물(강약점 분석)
    report: str        # Writer 산출물(최종 리포트)
```

`TeamState` 는 세 단계가 공유하는 서류철입니다(03번). 각 노드가 자기 칸만 채웁니다.

2. 입력 데이터 → 텍스트

경쟁사 CSV를 LLM이 읽기 좋은 한 덩어리 텍스트로 만듭니다.

```
def load_competitor_text() -> str:
    """경쟁사 CSV를 읽어 LLM에 넣을 한 덩어리 텍스트로 만든다."""
    df = pd.read_csv(DATA / "competitor_data.csv")
    lines = [
        f"- {r.company}: 주력={r.main_category}, 평점={r.rating}, "
        f"강점={r.strength}, 약점={r.weakness}"
        for r in df.itertuples()
    ]
    return "승승장구물 경쟁사 현황:\n" + "\n".join(lines)
```

`df.itertuples()` 로 각 행을 순회하며 `- 패속마켓: 주력=전자기기, 평점=4.3, ...` 형태로 만듭니다. 표를 LLM이 이해하기 쉬운 불릿 텍스트로 바꾸는 거죠.

3. 노드 = 에이전트 (세 역할 함수)

각 노드는 **state**를 받아 → 바꿀 칸만 **dict**로 반환합니다(03번 규칙).

```
# 각 노드는 'state를 받아 → 바꿀 칸만 dict로 반환'한다. LangGraph가 그 칸을 병합한다.
llm = get_chat(temperature=0.2) # 사실 정리·분석 위주라 낮은 온도

def researcher(state: TeamState) -> dict:
    """원천 데이터를 해석 없이 사실 위주로 정리한다(1단계)."""
    out = llm.invoke([
        SystemMessage("너는 리서처다. 주어진 데이터를 해석 없이 '사실'만 항목별로 깔끔히 정리하라."),
        HumanMessage(state["raw"]), # 최초엔 원천 텍스트가 들어 있음
    ]).content
    return {"raw": out} # 정리된 사실로 raw 칸 갱신 → 다음 노드가 사용

def analyst(state: TeamState) -> dict:
    """정리된 사실로 강점·약점·기회를 분석한다(2단계)."""
    out = llm.invoke([
        SystemMessage("너는 시장 분석가다. 경쟁사별 강점/약점과 승승장구물의 기회를 분석하라."),
        HumanMessage(f"[정리된 사실]\n{state['raw']}"), # 앞 노드가 채운 raw 를 입력으로
    ]).content
    return {"analysis": out} # analysis 칸 채우기

def writer(state: TeamState) -> dict:
    """분석 결과를 임원 보고용 마크다운 리포트로 작성한다(3단계)."""
    out = llm.invoke([
        SystemMessage("너는 보고서 작성자다. 분석 내용을 임원 보고용 마크다운 리포트로 작성하라. "
                       "제목, 요약, 경쟁사별 정리, 결론(전략 제언) 순으로."),
        HumanMessage(f"[분석 결과]\n{state['analysis']}"), # 앞 노드의 analysis 를 입력으로
    ]).content
    return {"report": out} # report 칸 채우기 = 최종 산출물
```

4. 세 노드의 공통 패턴

세 함수가 똑같은 모양임에 주목하세요.

```
def 노드(state):
    1) 앞 단계 산출물을 state에서 꺼냄 (입력)
    2) 역할 SystemMessage + 데이터 HumanMessage 로 llm.invoke (처리)
    3) 결과를 '자기 칸'만 dict로 반환 (출력)
```

- 입력이 다름: researcher는 `state["raw"]` (원천), analyst는 `state["raw"]` (정리된 사실), writer는 `state["analysis"]`. 앞 단계 결과만 받습니다(02번).
- **SystemMessage**가 역할을 고정: "너는 리서처다 / 분석가다 / 작성자다". 같은 LLM이지만 역할에 따라 다르게 일합니다.
- 반환이 자기 칸만: `{"raw": ...}`, `{"analysis": ...}`, `{"report": ...}`.

주의 — **researcher**의 입력도 `state["raw"]`: 최초 실행 때 `raw`에는 원천 텍스트가 들어 있고(05번 초기 상태), researcher가 그걸 정리된 사실로 덮어씁니다. 같은 칸을 "원천 → 정리본"으로 바꿔 쓰는 것이죠.

5. 왜 temperature=0.2인가

```
llm = get_chat(temperature=0.2)
```

이번 작업은 **사실 정리·분석** 위주라, 창의성보다 **일관성·정확성**이 중요합니다. 그래서 낮은 온도(0.2)로 둡니다. (Worker가 카피 작성에 0.5를 쓴 것과 대비 — 작업 성격에 맞춰 온도를 고릅니다.)

핵심 정리

- TeamState(공유 서류철)와 load_competitor_text(입력 로더)를 정의했다.
- 세 노드(researcher·analyst·writer)는 모두 "앞 칸 꺼내기 → llm.invoke → 자기 칸 반환" 패턴.
- SystemMessage로 역할을 고정, 입력은 앞 단계 산출물만.
- 사실·분석 위주라 **temperature=0.2**(일관성 우선).

따라하기 — StateGraph 로 연결

목표

04번에서 만든 세 노드를 **StateGraph**로 등록·연결해 실행 가능한 팀(그래프)으로 컴파일하고, `stream`으로 진행 상황을 보며 최종 리포트를 얻습니다.

1. 그래프 조립

세 노드를 등록하고 엣지로 순서를 잇습니다.

```
def build_team():
    """세 에이전트를 노드로 등록하고 순서대로 엣지를 이어 팀(그래프)을 컴파일한다."""
    g = StateGraph(TeamState)          # State 모양으로 그래프 시작
    g.add_node("researcher", researcher) # 노드 = 에이전트 등록
    g.add_node("analyst", analyst)
    g.add_node("writer", writer)

    g.add_edge(START, "researcher")    # 시작 → 리서처
    g.add_edge("researcher", "analyst") # 리서처 → 분석가
    g.add_edge("analyst", "writer")     # 분석가 → 작성자
    g.add_edge("writer", END)           # 작성자 → 끝
    return g.compile()                 # 실행 가능한 그래프로 컴파일
```

START → researcher → analyst → writer → END

흐름이 이 함수 한 곳에 그림처럼 선언됐습니다. 단계를 추가하거나 순서를 바꾸려면 여기 엣지만 고치면 되죠.

- `add_node("이름", 함수)` — 노드 등록. 첫 인자는 그래프 안에서 부를 이름.
- `add_edge(A, B)` — "A 다음엔 B" 라는 연결.
- `START` / `END` — 그래프의 진입·종료 지점(LangGraph 내장 상수).
- `g.compile()` — 그래프를 실행 가능한 객체로 만듦.

2. 실행 — stream으로 진행 보기

```
def main():
    team = build_team()
    # 초기 상태: raw 에 원천 텍스트를 넣고, 나머지 칸은 빈 문자열로 시작
    initial = {"raw": load_competitor_text(), "analysis": "", "report": ""}

    print("AI 팀 가동: Researcher → Analyst → Writer\n")
    final = initial
    for state in team.stream(initial, stream_mode="values"):
        final = state          # 매번 최신 누적 상태를 잡아둠(마지막이 최종 결과)
        done = [k for k in ("analysis", "report") if state.get(k)]
        print(f" [진행] 누적 상태: {'', '.join(done) if done else '시작'}")

    print("\n" + "=" * 60)
    print(final["report"])

if __name__ == "__main__":
    main()
```

초기 상태가 중요합니다. `raw` 에 원천 텍스트를 넣고, 아직 안 채워진 `analysis` · `report` 는 빈 문자열로 시작합니다. 그래프가 돌면서 빈 칸들이 차례로 채워집니다.

3. stream_mode="values" — 진행과 결과를 한 번에

```
for state in team.stream(initial, stream_mode="values"):
    final = state
```

`stream(stream_mode="values")` 는 매 노드 실행 후 누적된 전체 상태를 하나씩 내보냅니다.

```
researcher 끝 → {raw: 정리본, analysis: "",    report: ""}
analyst 끝   → {raw: 정리본, analysis: 분석,   report: ""}
writer 끝    → {raw: 정리본, analysis: 분석,   report: 리포트} ← 마지막 = 최종
```

`final = state` 로 매번 최신 상태를 잡아두면, 루프가 끝났을 때 `final` 에 최종 결과가 들어 있습니다.

⚠ 왜 **stream**을 한 번만 쓰나 — 진행 상황은 `stream` 으로 보고 결과는 `invoke` 로 또 받으면, 파이프 라인이 두 번 실행되어 LLM 호출 비용이 그대로 2배가 됩니다. 그래서 `final` 변수에 마지막 상태를 잡아두는 방식으로 딱 한 번만 실행합니다. (실무 비용 감각 — 2강 토큰·비용.)

4. 실행 결과

AI 팀 가동: Researcher → Analyst → Writer

[진행] 누적 상태: 시작
[진행] 누적 상태: 시작
[진행] 누적 상태: analysis
[진행] 누적 상태: analysis, report

승승장구물 경쟁사 분석 리포트

요약

쾌속마켓은 당일배송이 강점이나 가격 경쟁력이 약하고, 알뜰몰은 최저가가 강점이나 배송이 느립니다 ...

경쟁사별 정리

- **쾌속마켓** (평점 4.3): 강점 당일배송 / 약점 가격 약간 높음 ...
- **알뜰몰** (평점 4.0): 강점 최저가 정책 / 약점 배송 느림 ...

결론 — 전략 제언

1. 가격 민감 고객을 겨냥한 ...

관찰 포인트

- [진행] 이 시작 → 시작 → analysis → analysis, report 로 변합니다. 빈 칸이 차례로 채워지는 모습이죠.
- (처음 두 줄이 "시작"인 이유: 초기 상태 1번 + researcher가 raw 만 채운 상태 1번 — analysis · report 는 아직 비어 있어 둘 다 "시작"으로 표시됩니다.)
- 세 노드는 공유 State를 통해서만 데이터를 주고받습니다(느슨한 결합).
- 경쟁사 CSV 한 장에서 완성된 분석 리포트가 나왔습니다.

5. 단계를 바꿔보기 — 그래프의 장점

만약 Analyst 앞에 'Critic(비평가)' 노드를 넣고 싶다면?

```
g.add_node("critic", critic)
g.add_edge("researcher", "critic") # researcher → critic
g.add_edge("critic", "analyst")   # critic → analyst (기존 researcher→analyst 대체)
```

그래프(엣지)만 수정하면 됩니다. 노드 함수들은 그대로죠. 흐름이 한 곳에 선언돼 있어 확장이 명확합니다. 06번에서는 여기에 조건부 분기(검수 후 되돌리기)를 더해, "왜 그래프인가"를 제대로 체감합니다.

핵심 정리

- build_team(): add_node 로 노드 등록 → add_edge 로 순서 연결 → compile().
- 초기 상태는 raw 에 **원천 텍스트**, 나머지 칸은 빈 문자열로 시작.
- stream(stream_mode="values"): 매 노드 후 **누적 상태**를 내보냄 → final 에 잡아두면 최종 결과.
- **stream** 은 **한 번만** — invoke 를 또 부르면 파이프라인이 2 번 돌아 비용 2 배.
- 단계 추가·순서 변경은 **그래프(엣지)**만 고치면 됨.

조건부 분기로 품질 재검토

지금까지의 파이프라인은 **한 방향 일직선**이었습니다.

여기에 **검수(check)** 노드와 **갈림길(add_conditional_edges)** 을 더해, 분석이 **부실하면 다시 조사(루프)**, **충분하면 작성으로** 진행하게 만듭니다.

무한루프를 막도록 재시도 횟수를 상태에 두고 **최대 2 회**로 제한합니다. 여기서 비로소 **"왜 그래프인가"** 가 드러납니다.

1. 일직선의 한계 — 부실해도 그냥 통과

05번 파이프라인은 분석이 좋은 나쁜든 무조건 **Writer**로 넘어갑니다.

researcher → analyst → writer (분석이 부실해도 그대로 보고서行)

분석이 두 줄밖에 안 나오거나 강점·약점이 빠져도, Writer는 그 부실한 분석으로 보고서를 씁니다. "분석이 별로면 다시 조사하라" 는 판단이 없죠. 이걸 더하려면 **갈림길**이 필요합니다.

2. 목표 흐름 — 검수 후 갈림길

researcher → analyst → **check** 에서 갈립니다.

- (부실 & 재시도 여유) → **redo** → analyst ...(루프)
- (충분 or 한도 초과) → **writer** → END
- **check(검수)**: 분석 품질을 평가해 **충분** / **부실** 판정.

- 부실 + 재시도 여유 → `redo` (재조사)로 갔다가 다시 `analyst` 로 (루프).
- 충분 or 재시도 한도 초과 → `writer` 로 진행.

이 "갈림길 + 되돌아가기"는 `for` / `if` 로 짜면 엉키지만, 그래프에선 엣지 한 줄로 선언됩니다.

3. 상태 확장 — 카운터와 판정 칸

기존 `TeamState` 에 재시도 횟수와 판정을 담은 칸을 추가합니다.

```
from ch27_ai_team import (                # 기존 노드 · 로더 그대로 재사용
    TeamState as BaseState,
    researcher, analyst, writer, load_competitor_text,
)

class TeamState(BaseState):    # 기존 상태를 상속해 칸 추가
    retries: int    # 재수집(루프) 횟수 — 무한루프 방지용 카운터
    verdict: str    # 검수 결과: '충분' | '부실'

MAX_RETRIES = 2    # [경고] 루프 상한 — 넘으면 부실해도 Writer로 강제 진행
```

`retries`(카운터) 가 핵심입니다.

루프를 돌 때마다 1 씩 올려, **상한(2 회)** 에 닿으면 더 안 돌게 합니다.

무한루프를 막는 안전장치죠 (`max_steps` 정신).

4. 검수 노드 — 품질 평가

분석이 충분한지 규칙 기반으로 빠르게 거릅니다.

```
def check(state: TeamState) -> dict:
    """Analyst 산출물의 품질을 평가하는 검수 노드."""
    analysis = state.get("analysis", "")
    # 규칙 기반 1차 필터: 너무 짧거나 핵심 키워드가 없으면 '부실'
    too_short = len(analysis) < 200
    missing_core = not any(k in analysis for k in ("강점", "약점", "기회"))
    verdict = "부실" if (too_short or missing_core) else "충분"
    print(f" [검수] 분석 길이={len(analysis)}자, "
          f"핵심키워드={'없음' if missing_core else '있음'} → 판정: {verdict}")
    return {"verdict": verdict}
```

여기선 길이(200자 미만) 와 핵심 키워드(강점/약점/기회) 유무로 판정합니다. LLM에게 평가시킬 수도 있지만(비용 ↑), 간단한 규칙으로도 1차 거르기엔 충분합니다. (25강 규칙 라우터의 정신 — 싸고 빠른 규칙 먼저.)

5. 라우터 — 갈림길의 방향 결정

검수 결과에 따라 다음에 갈 노드 이름을 문자열로 반환하는 함수입니다.

```
def route_after_check(state: TeamState) -> str:
    """검수 결과에 따라 다음 노드 이름을 반환. add_conditional_edges가 이 값으로 분기."""
    # [경고] 무한루프 방지: 재시도 상한에 도달하면 부실이어도 진행
    if state["verdict"] == "부실" and state["retries"] < MAX_RETRIES:
        return "redo"
    return "ok"
```

두 조건을 모두 봅니다: **부실** 이면서 아직 재시도 여유(`retries < 2`)가 있을 때만 **redo**. 둘 중 하나라도 아니면(**충분** 이거나 한도 초과) **ok**. 이 함수가 갈림길의 판단입니다.

6. 재조사 노드 — 카운터 올리며 루프

```
def researcher_retry(state: TeamState) -> dict:
    """재수집 단계. 재시도 카운터를 올린 뒤 다시 사실을 정리한다."""
    n = state["retries"] + 1
    print(f" [재수집] {n}회차 — 부실 판정으로 Researcher 재가동")
    upd = researcher({"raw": load_competitor_text()}) # 원천부터 다시 정리
    upd["retries"] = n                               # 카운터 갱신
    return upd
```

원천 텍스트로 되돌려 **researcher** 를 다시 돌리고, **retries** 를 +1 합니다. 이 카운터 갱신이 없으면 무한루프에 빠지니 반드시 올려야 합니다.

7. 조건부 엣지로 그래프 조립

핵심은 `add_conditional_edges` 입니다.

```
def build_team():
    g = StateGraph(TeamState)
    g.add_node("researcher", researcher)
    g.add_node("analyst", analyst)
    g.add_node("check", check)
    g.add_node("redo", researcher_retry) # 재수집 노드(루프 경로)
    g.add_node("writer", writer)
```

```

g.add_edge(START, "researcher")
g.add_edge("researcher", "analyst")
g.add_edge("analyst", "check")          # 분석 다음엔 반드시 검수
# [핵심] 검수 후 갈림길: 라우터 반환값 → 실제 노드 매핑
g.add_conditional_edges(
    "check",
    route_after_check,                  # 이 함수가 "redo"/"ok" 반환
    {"redo": "redo", "ok": "writer"}, # 반환값 → 갈 노드
)
g.add_edge("redo", "analyst")          # 재수집 후 다시 분석 → 다시 검수(루프!)
g.add_edge("writer", END)
return g.compile()

```

add_conditional_edges(노드, 라우터함수, 매핑):

```

check 노드 끝 → route_after_check(state) 호출 → "redo" 또는 "ok" 반환
"redo" → redo 노드로
"ok"   → writer 노드로

```

그리고 **redo → analyst** 엣지가 루프를 만듭니다: **analyst → check → redo → analyst → check → ...** 가 검수를 통과하거나 한도에 달을 때까지 돕니다.

8. 실행과 결과

```

if __name__ == "__main__":
    team = build_team()
    initial = {"raw": load_competitor_text(), "analysis": "", "report": "",
               "retries": 0, "verdict": ""}      # 카운터 · 판정 칸도 초기화
    final = initial
    for state in team.stream(initial, stream_mode="values"):
        final = state
    print(f"\n최종 재시도 횟수: {final['retries']}회 / 최종 판정: {final['verdict']}")
    print(final["report"])

```

예상 출력

[검수] 분석 길이=512자, 핵심키워드=있음 → 판정: 충분

최종 재시도 횟수: 0회 / 최종 판정: 충분
승승장구물 경쟁사 분석 리포트 ...

분석이 충분하면 **재시도 0회**로 바로 통과합니다. 만약 부실했다면 **[재수집] 1회차 ...**가 찍히며 루프를 돌고, 그래도 안 되면 2회에서 멈춰 강제 진행하죠.

9. 여기서 드러나는 "왜 그래프인가"

[for/if 로 찼다면]

while + 카운터 + 분기 조건이 main 코드에 뒤엉킴 → 흐름이 안 보임

[StateGraph]

check · redo · writer는 각자 함수, 흐름은 build_team()의 엣지에 '그림'으로 선언

→ "검수 후 부실이면 redo로 돌고, 아니면 writer로" 가 한눈에 보임

되돌아가기·반복·조건 분기 — 바로 이때 StateGraph의 진가가 나옵니다. 일직선이면 `for`로 충분했지만, 갈림길이 생기니 그래프가 흐름을 훨씬 명확하게 표현합니다.

핵심 정리

- 일직선 파이프라인에 **검수 노드 + 조건부 분기**를 더해 품질 재검토 루프를 만들.
- `add_conditional_edges`(노드, 라우터, 매핑): 라우터 반환값으로 **다음 노드를 선택**.
- `redo` → `analyst` 엣지가 **루프**를 형성, `retries` 카운터 + `MAX_RETRIES`로 **무한루프 방지**.
- 검수는 **규칙 기반**(길이·키워드)으로 싸고 빠르게 1차 거르기.
- **되돌아가기·반복이 생길 때** StateGraph의 진가가 드러난다.

중간 산출물 저장

파이프라인이 만든 **중간 산출물**(정리된 사실·분석·리포트)을 화면에만 띄우고 버리지 말고, **파일로 남깁니다**.

디스크에 저장하면 **디버깅·재현·감사**(누가 무엇을 만들었나)에 큰 도움이 됩니다. 각 노드를 감싸는 **저장 래퍼 노드**로 깔끔하게 구현합니다.

1. 왜 중간 산출물을 저장하나

지금까지는 최종 `report`만 화면에 출력하고 끝이었습니다. 중간 결과(`raw`, `analysis`)는 사라지죠. 하지만 실무에선 중간물이 중요합니다.

- 디버깅: 리포트가 이상하다 → 분석이 문제였나? 사실 정리가 문제였나?
중간물이 있어야 어느 단계에서 틀렸는지 추적 가능
- 재현: 같은 입력으로 다시 안 돌려도, 저장된 중간물을 재사용
- 감사: "이 리포트는 어떤 사실·분석을 근거로 나왔나"를 파일로 증빙
- 운영: 산출물을 로그처럼 쌓아두는 운영 자산

특히 LLM 파이프라인은 **매 실행이 비용**이라, 중간물을 저장해두면 다시 안 돌려도 됩니다.

2. 저장 함수 — 타임스탬프 파일로

산출물 한 건을 `reports/` 폴더에 타임스탬프 이름으로 저장합니다.

```
import datetime
from common import ROOT          # 프로젝트 루트 경로 (common.py 제공)

REPORTS = ROOT / "reports"      # 산출물을 코드와 분리해 한 폴더에 모음

def save_artifact(name: str, content: str, ext: str = "md") -> pathlib.Path:
    """산출물 한 건을 reports/ 아래 타임스탬프 파일로 저장하고 경로를 반환한다."""
    REPORTS.mkdir(parents=True, exist_ok=True)    # reports/ 자동 생성
    stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    path = REPORTS / f"{stamp}_{name}.{ext}"
    path.write_text(content, encoding="utf-8")    # 한글 안전하게 utf-8
    print(f" [저장] {name} -> {path}")
    return path
```

- `REPORTS.mkdir(parents=True, exist_ok=True)` — 폴더가 없으면 만들고, 있으면 그냥 넘어감(에러 안 남).
- `datetime.now().strftime("%Y%m%d_%H%M%S")` — `20260607_143022` 같은 시각 도장. 실행마다 다른 파일명이 되어 덮어쓰기 방지.
- `write_text(..., encoding="utf-8")` — 한글이 깨지지 않게 인코딩 지정.

산출물을 코드와 섞지 않고 `reports/` 한 폴더에 모으는 게 포인트입니다. 로그처럼 쌓이는 운영 자산으로 관리하는 거죠.

3. 저장 래퍼 노드 — 기존 노드를 감싸기

기존 노드를 수정하지 않고, 실행 후 저장만 덧붙이는 래퍼 노드를 만듭니다.

```
from ch27_ai_team import (          # 기존 노드 그대로 재사용
    TeamState as BaseState,
    researcher, analyst, writer, load_competitor_text,
)
```

```
def researcher_save(state: BaseState) -> dict:
    """Researcher 실행 → 정리된 사실(raw)을 .txt로 저장."""
    upd = researcher(state)          # 기존 노드 그대로 실행
    save_artifact("01_raw", upd["raw"], ext="txt")  # 산출물 저장
    return upd

def analyst_save(state: BaseState) -> dict:
    """Analyst 실행 → 분석 결과(analysis)를 .md로 저장."""
    upd = analyst(state)
    save_artifact("02_analysis", upd["analysis"])
    return upd

def writer_save(state: BaseState) -> dict:
    """Writer 실행 → 최종 리포트(report)를 .md로 저장."""
    upd = writer(state)
    save_artifact("03_report", upd["report"])
    return upd
```

설계 포인트 — 각 래퍼는 **기존 노드 실행 → 결과 저장 → 결과 반환**. 기존 `researcher` / `analyst` / `writer`는 건드리지 않습니다. 저장이라는 부가 기능을 덧입히는 거죠(데코레이터 패턴의 정신). 파일명에 `01_`, `02_`, `03_` 접두사를 붙여 단계 순서를 드러냅니다.

4. 그래프 조립과 실행

흐름은 27강 원본과 똑같고, 노드만 저장 래퍼로 바꿉니다.

```
def build_team():
    """저장 래퍼 노드로 묶은 파이프라인(흐름은 27강과 동일)."""
    g = StateGraph(BaseState)
    g.add_node("researcher", researcher_save)  # 래퍼 노드 등록
    g.add_node("analyst", analyst_save)
    g.add_node("writer", writer_save)

    g.add_edge(START, "researcher")
    g.add_edge("researcher", "analyst")
    g.add_edge("analyst", "writer")
    g.add_edge("writer", END)
    return g.compile()
```

```

if __name__ == "__main__":
    team = build_team()
    initial = {"raw": load_competitor_text(), "analysis": "", "report": ""}
    print(f"저장 위치: {REPORTS}")
    final = initial
    for state in team.stream(initial, stream_mode="values"):
        final = state
    print(final["report"])

```

예상 출력

```

저장 위치: /.../reports
[저장] 01_raw → /.../reports/20260607_143022_01_raw.txt
[저장] 02_analysis → /.../reports/20260607_143041_02_analysis.md
[저장] 03_report → /.../reports/20260607_143103_03_report.md
# 승승장구물 경쟁사 분석 리포트 ...

```

실행 후 reports/ 폴더에 **3 개 파일**이 단계별로 쌓입니다. 리포트가 이상하면 02_analysis.md 를 열어 "분석이 부실했나"를 바로 확인할 수 있죠.

5. 응용 — 한 폴더로 묶기

실행마다 타임스탬프가 달라 파일이 흩어진다면, 실행 단위 폴더로 묶을 수도 있습니다.

```

# 한 번 실행 = 한 폴더 (선택적 개선)
run_dir = REPORTS / datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
run_dir.mkdir(parents=True, exist_ok=True)
# 이후 save_artifact가 run_dir 아래에 01_raw, 02_analysis, 03_report 저장

```

이러면 reports/20260607_143022/ 안에 그 실행의 산출물 3 개가 모입니다. "이 실행의 결과물 묶음"을 한눈에 보기 좋죠. (운영 로깅과 연결되는 발상입니다.)

핵심 정리

- 중간 산출물을 **파일로 저장**하면 디버깅·재현·감사·운영에 유리.
- save_artifact: reports/에 **타임스탬프 파일**로 저장(mkdir(exist_ok=True), utf-8).
- **저장 래퍼 노드**: 기존 노드를 감싸 실행 → 저장 → 반환 — 원본 노드는 안 건드림.
- 흐름(그래프)은 원본과 동일, **노드만 래퍼로 교체**.

3. Project design

바로 코딩이 망하는 이유

이전까지 도구·RAG·메모리·멀티에이전트를 **따로따로** 만들었습니다.

이번 장에선 이걸 **하나의 서비스**로 합칩니다. 그런데 **설계 없이 바로 코드부터** 짜면, 책임이 뒤섞이고 인터페이스가 불명확하며 비용·실패 대비가 빠진 결과가 나옵니다.

에이전트도 결국 **하나의 백엔드 서비스** — **설계 → 구현** 순서를 지켜야 합니다.

1. 이제 "합치는" 단계입니다

지금까지 배운 것들을 떠올려 봅시다.

5~12강: 도구(주문조회 · 재고 · 계산 · 검색)
13~18강: RAG(정책 문서 검색)
19~20강: 메모리(대화 맥락 · 사용자 기억)
21~27강: 워크플로우 · 멀티에이전트(협업)

각각은 **따로** 동작했습니다. 30강의 목표는 이걸 모아 "**진짜로 일하는 승승장구물 통합 CS 에이전트**" 하나로 만드는 것입니다. 그런데 여기서 많은 사람이 실수합니다 — **곧장 코드부터** 짜기 시작하죠.

2. "바로 코딩"이 부르는 참사

설계 없이 바로 짜면, 백엔드 프로젝트가 그렇듯 이런 문제가 터집니다.

- ① 책임이 뒤섞임
 - 주문 조회 코드 안에 RAG 검색이 끼어들고, 답변 생성에 DB 로직이 섞임
 - "이 기능 고치려면 어디를 봐야 하지?"가 안 보임
- ② 인터페이스가 불명확
 - 함수 이름 · 입출력이 매번 달라, 나중에 FastAPI로 감쌀 때 고생
 - "이 에이전트를 외부에서 어떻게 부르지?"가 안 정해짐
- ③ 비용 · 실패 대비 누락
 - LLM 호출이 몇 번 일어나는지, 실패하면 어쩔지 생각 안 함
 - 운영에서 타임아웃 · rate limit 만나면 그대로 멈춤
- ④ "필요 없는 것"까지 넣음
 - 도구 4개뿐인데 멀티에이전트로 거창하게 → 복잡도 · 비용만 폭증

특히 ④가 혼합니다. "멋져 보여서" RAG·멀티에이전트를 다 넣었다가 오히려 느리고 비싸고 디버깅 어려운 시스템이 되죠.

3. 에이전트도 "백엔드 서비스"다

핵심 인식 전환입니다.

에이전트 = 입력을 받아 처리해 출력을 내는 하나의 백엔드 서비스.

백엔드 서비스를 설계 없이 만들면 운영에서 터지듯, 에이전트도 똑같습니다. 그래서 **설계 → 구현** 순서를 지켜야 합니다.

[바로 코딩] 코드부터 → 중간에 구조가 흔들림 → 갈아엎기 → 시간 낭비
[설계 먼저] 무엇을 · 어떻게 만들지 합의 → 구현은 채우기만 → 흔들림 없음

4. 이번 강의 하는 일 — 코드 대신 "설계 문서"

28강은 코드를 거의 안 짭니다. 대신 설계 문서를 만듭니다. 그리고 그 설계를 코드 스켈레톤(시그니처 + **TODO**) 으로 박제합니다.

이번 강의 결과물:

- "승승장구물 통합 CS 에이전트" 설계 문서
- ch28_design.py — 함수 시그니처 + TODO 만 있는 '살아있는 설계 문서'
(실행하면 설계 요약출력, 실제 LLM 호출은 30강에서)

이번 장에서는 이 스켈레톤의 **TODO**를 채우기만 하면 됩니다. 설계가 미리 고정돼 있어 구현이 흔들리지 않죠.

5. 5단계 설계 절차 (이번 강의 뼈대)

이번 강은 다음 5단계를 따라갑니다.

- | | | |
|------------------|------------------------------------|---------------|
| 1단계, 요구사항 정의 | — "무엇을 해결하나" (사용자 시나리오) | → 02번 |
| 2단계, 기능 분해 | — 요구사항을 함수 단위로 쪼갬 | → 02번 |
| 3단계, 구성요소 필요성 판단 | — 도구 · RAG · 메모리 · 멀티에이전트 중 뭐가 필요? | → 03번 (가장 중요) |
| 4단계, 아키텍처 설계 | — 전체 그림(다이아그램) | → 04번 |
| 5단계, 인터페이스 설계 | — 외부 계약(진입 함수 · 엔드포인트) | → 04번 |

이 절차는 에이전트뿐 아니라 모든 백엔드 설계에 통하는 사고법입니다. 특히 3단계(구성요소 필요성 판단)가 핵심 — "필요 없는 건 안 넣는다"가 좋은 설계의 출발입니다.

핵심 정리

- 이번 강에서 도구·RAG·메모리·멀티에이전트를 하나의 서비스로 통합한다.
- 설계 없이 바로 코딩하면: 책임 혼재·인터페이스 불명확·비용/실패 누락·과잉 구성.
- 에이전트도 하나의 백엔드 서비스 — 설계 → 구현 순서를 지켜야 한다.
- 이번 강은 코드 대신 설계 문서 + 코드 스켈레톤(살아있는 설계 문서)을 만든다.
- 5단계 절차: 요구사항 → 기능 분해 → 구성요소 판단 → 아키텍처 → 인터페이스.

요구사항 정의와 기능 분해 (1·2단계)

설계의 첫 두 단계입니다. 1단계 요구사항 정의 — "무엇을 해결하나"를 사용자 시나리오로 적습니다. 2단계 기능 분해 — 각 요구사항을 함수 단위(입력/출력)로 쪼갭니다. 막연한 "CS 봇 만들기"를 구체적인 함수 목록으로 바꾸는 과정입니다.

1. 1단계 — 요구사항 정의 (사용자 시나리오)

"무엇을 만들지"를 기능이 아니라 사용자가 실제로 하는 말로 적습니다. 승승장구를 통한 CS 에이전트라면:

#	사용자 시나리오	필요한 능력
R1	"환불 절차 알려줘"	사내 정책 문서 검색(RAG)
R2	"내 주문 O000050 상태는?"	주문 DB 도구
R3	"이어버드 재고 있어?"	재고 도구
R4	"무료배송 기준?"	FAQ 도구
R5	"아까 그 정책 다시 알려줘"	단기 메모리(대화 맥락)

왜 사용자 시나리오로 시작하나? "기능 목록"부터 적으면 만들기 쉬운 것 위주로 흐릅니다. 반면 사용자가 실제로 하는 말에서 출발하면, 진짜 필요한 능력이 드러나죠. R1~R5는 "고객이 상담창에 칠 법한 문장"이고, 각 문장이 곧 시스템이 갖춰야 할 능력을 가리킵니다.

"LLM이 답 못 한 질문 2(주문 상태)"를 기억하시나요? 그게 바로 여기 **R2**입니다. 이 장에서 드디어 이걸 풀니다.

2. 2단계 — 기능 분해 (요구사항 → 함수)

각 요구사항을 함수 단위로 쪼갭니다. 입력과 출력을 명시해, "무엇을 받아 무엇을 내는가"를 못박습니다.

```
get_order_status(order_id) -> 주문 상태 문자열      (R2)
get_stock(product_name)   -> 재고 수량 문자열        (R3)
search_faq(keyword)       -> FAQ 답변                (R4)
policy_search(query)       -> 정책 문서 근거(RAG)     (R1)
+ 대화 thread_id 별 메모리                               (R5)
```

[요구사항]	→	[함수 (입력 → 출력)]
R1 정책 안내	→	policy_search(query) → 근거 텍스트
R2 주문 상태	→	get_order_status(order_id) → 상태 문자열
R3 재고 확인	→	get_stock(product_name) → 재고 문자열
R4 FAQ	→	search_faq(keyword) → 답변
R5 대화 기억	→	(함수 아님) thread_id 별 메모리로 처리

이건 백엔드의 유스케이스 분해와 똑같습니다. "주문 상태 알려줘"라는 추상적 요구를,

`get_order_status(order_id) -> str` 라는 구체적 계약으로 바꾸는 거죠.

3. 왜 입력/출력을 먼저 정하나

함수의 시그니처(이름·입력·출력)를 먼저 정하면 좋은 점:

- 합의 고정: "주문 조회는 order_id를 받아 문자열을 낸다"가 못박힘
→ 구현자마다 제각각 만드는 일이 없음
- 도구화 준비: 5강에서 배운 @tool 은 함수의 시그니처 · 독스트링을 LLM이 읽음
→ 시그니처가 명확해야 LLM이 도구를 잘 고름
- 테스트 용이: 입출력이 정해져 있어, 구현 전에 테스트부터 짤 수 있음
- 병렬 작업: 팀이면 시그니처만 합의하고 각자 구현 가능

특히 에이전트에서는 이 함수들이 곧 도구(@tool)가 됩니다. 시그니처와 독스트링이 명확해야 LLM이 "이 질문엔 이 도구"를 제대로 고르죠(6강 도구 설계 원칙).

4. 함수로 안 쪼개지는 것 — R5(메모리)

주의할 점: R5(대화 기억)는 함수가 아닙니다.

R1~R4: 입력 → 출력이 명확한 '함수(도구)'

R5: "아까 그 정책" 을 이해하려면 → 대화 맥락을 유지하는 '메모리'

"아까 그거 다시 알려줘"는 특정 입력→출력 함수로 만들 수 없습니다. 이전 대화를 기억하는 장치(메모리)가 필요하죠. 그래서 R5는 도구가 아니라 `thread_id` 별 단기 메모리(19강)로 처리합니다.

이렇게 요구사항을 분해하다 보면 "이건 도구, 저건 메모리, 그건 RAG" 가 자연스럽게 갈립니다. 그 판단을 체계적으로 하는 게 다음 절(3단계, 구성요소 필요성 판단)입니다.

5. 분해 결과 정리

지금까지의 결과를 모으면:

도구 4개: `get_order_status`, `get_stock`, `search_faq`, `policy_search`
RAG: `policy_search` 가 정책/멤버십 PDF를 검색 (R1)
메모리: `thread_id` 별 단기 메모리 (R5)
멀티에이전트: ? (아직 판단 안 함 — 3단계에서)

요구사항 5개가 구체적인 구성요소 목록으로 바뀌었습니다. "CS 봇 만들기"라는 막연한 목표가, 이제 "도구 4개 + RAG + 메모리"라는 만들 수 있는 단위가 됐죠.

핵심 정리

- 1단계(요구사항): "무엇을 해결하나"를 사용자 시나리오(R1~R5)로 적는다 — 기능이 아니라 사용자의 말에서 출발.
- 2단계(기능 분해): 각 요구사항을 함수(입력→출력)로 쪼갬다 — 백엔드 유스케이스 분해와 동일.
- 시그니처를 먼저 정하면 합의 고정·도구화·테스트·병렬작업에 유리.
- R5(메모리)는 함수로 안 쪼개짐 → `thread_id` 별 단기 메모리로 처리.

구성요소 필요성 판단 (3단계 — 가장 중요)

설계에서 **가장 중요한 단계**입니다. 도구·RAG·단기메모리·장기메모리·멀티에이전트 중 무엇이 정말 필요한지를 판단 기준으로 결정합니다. 핵심 원칙은 단 하나 — **필요 없는 구성요소는 넣지 않는다**. "멋져 보여서" 넣으면 복잡도·비용만 늘어납니다.

1. "다 넣으면 좋은 거 아냐?" — 아닙니다

흔한 오해입니다. RAG도 좋고 멀티에이전트도 멋지니 다 넣으면 강력하지 않을까?

[다 넣은 시스템]

도구 + RAG + 장기메모리 + 멀티에이전트 + 워크플로우 + ...
→ LLM 호출 폭증(비용↑) + 지연↑ + 디버깅 지옥 + 유지보수 부담
→ 정작 도구 4개로 충분했던 일을 거창하게 만들

[필요한 것만 넣은 시스템]

도구 4개 + RAG + 단기메모리
→ 단순 · 빠름 · 저렴 · 디버깅 쉬움

좋은 설계는 더하는 게 아니라 빼는 것입니다. 각 구성요소는 그만큼 **값(복잡도·비용)** 을 치릅니다. 그 값을 치를 만큼 꼭 필요한지 따져야 하죠.

2. 판단 기준표

각 구성요소를 "이럴 때 필요하다"는 기준으로 판단합니다.

구성요소	"필요하다"고 판단하는 기준	우리 케이스
Tool(도구)	실시간·구조화 데이터 조회/행동이 필요	✅ 주문/재고/FAQ
RAG	양 많고 자주 바뀌는 문서를 근거로 답해야 함	✅ 정책/멤버십 PDF
단기 메모리	한 대화 안에서 이전 발화를 참조	✅ "아까 그거"
장기 메모리	세션을 넘어 사용자별 정보 기억	❌ (이번 범위 밖)
멀티에이전트	도구 5~7개 초과 또는 책임 충돌	△(도구 4개 → 단일로 시작)

각 결정을 하나씩 봅시다.

3. 도구(Tool) — 필요

기준: 실시간·구조화 데이터 조회나 행동이 필요한가?

주문 상태(R2)·재고(R3)·FAQ(R4)는 모두 **DB/CSV에 있는 실시간·구조화 데이터**입니다. LLM이 지어낼 수 없고 조회해야 하죠. → 도구 필요. (1강에서 "LLM이 답 못 한" 바로 그 이유)

4. RAG — 필요

기준: 양 많고 자주 바뀌는 문서를 근거로 답해야 하는가?

환불·멤버십 정책(R1)은 **PDF 문서**에 길게 적혀 있고, 회사 사정에 따라 **바뀝니다**. 이걸 프롬프트에 다 욱여 넣을 수 없고, 지어내서도 안 됩니다. → 문서를 검색해 근거로 답하는 RAG 필요.

RAG가 맞는 경우: 긴 문서 · 자주 바뀜 · 근거(출처) 필요 → 정책 PDF
RAG가 과한 경우: 답이 고정적이고 짧음 → 그냥 프롬프트에 넣으면 됨
실시간 데이터: RAG 아님! → 도구로 조회 (주문 · 재고는 문서가 아니라 DB)

주의 — "실시간 데이터(주문·재고)"는 RAG가 아니라 도구입니다. RAG는 고정된 문서를 검색하는 것이지, 실시간으로 변하는 DB 값을 가져오는 게 아닙니다. 이 구분이 자주 헷갈립니다.

5. 단기 메모리 — 필요 / 장기 메모리 — 이번엔 불필요

단기 기준: 한 대화 안에서 이전 발화를 참조하는가?

장기 기준: 세션을 넘어 사용자별 정보를 기억해야 하는가?

"아까 그 정책 다시"(R5)는 같은 대화 안의 맥락 참조라 → 단기 메모리(thread_id, 19강)면 충분합니다.

반면 "이 사용자가 한 달 전에 뭘 샀는지" 같은 세션을 넘는 기억은 이번 범위가 아닙니다. → 장기 메모리는 불필요. (필요해지면 20강 SQLite 방식으로 나중에 추가)

단기 메모리: 한 대화 세션 = thread_id 별 →  (R5)
장기 메모리: 세션 넘는 사용자 프로필 →  (이번엔 범위 밖, 과잉)

6. 멀티에이전트 — △ 단일로 시작

기준: 도구 5~7개 초과이거나 책임이 충돌하는가?

우리는 도구가 4개뿐입니다. 단일 `create_agent` 하나가 충분히 다룰 수 있죠. 굳이 역할을 나눠 멀티에이전트로 만들면 복잡도·비용만 늘어납니다.

멀티에이전트가 맞는 때: 도구 8개+ / 역할이 뚜렷이 갈림(리서처 · 분석가 · 작성자)
우리 케이스: 도구 4개, 단일 책임 → 단일 에이전트로 시작
(나중에 도구가 늘면 그때 분리해도 늦지 않음)

설계 원칙 재확인: 멀티에이전트는 도구가 적으면 오히려 손해입니다. 단일로 시작하고, 정말 복잡해지면 그때 나눈다. "나중에 나누기"는 쉽지만, 처음부터 과하게 나누면 되돌리기 어렵습니다.

7. 판단 결과 — 설계 확정

3단계를 거친 결론:

- ✓ 도구 4개 : `get_order_status`, `get_stock`, `search_faq`, `policy_search`
- ✓ RAG : 정책/멤버십 PDF 검색 (`policy_search` 내부)
- ✓ 단기메모리 : `thread_id` 별 대화 세션
- ✗ 장기메모리 : 이번 범위 밖
- ✗ 멀티에이전트: 단일 `create_agent` 로 시작 (도구 4개)

"무턱대고 다 넣기" 대신 기준으로 따져 넣을 것과 뺄 것을 정했습니다. 이 의사결정 과정 자체가 설계의 핵심이고, 06번에서는 이 판단을 규칙으로 자동화한 도우미를 만들어 봅니다.

핵심 정리

- 3단계는 가장 중요 — 도구·RAG·메모리·멀티에이전트 중 무엇이 필요한지 기준으로 판단.
- 핵심 원칙: 필요 없는 건 넣지 않는다(각 구성요소는 복잡도·비용을 치름).
- 도구=실시간·구조화 데이터, RAG=긴/변하는 문서 근거, 단기메모리=한 대화 맥락.
- 실시간 데이터는 RAG가 아니라 도구 (자주 헛갈리는 구분).
- 멀티에이전트는 도구 5~7개 초과/책임 충돌 시 — 우리는 4개라 단일로 시작.

아키텍처와 인터페이스 설계 (4·5단계)

설계의 마지막 두 단계입니다. **4단계 아키텍처** — **확정한 구성요소들이 어떻게 연결되는지** 전체 그림을 그립니다.

5단계 인터페이스 — 에이전트를 서비스로 쓰기 위한 **외부 계약**(진입 함수·세션·엔드포인트·실패 대비)을 정합니다.

1. 4단계 — 아키텍처 다이어그램

3단계에서 **확정한 구성요소(도구 4개 + RAG + 단기메모리)**가 **어떻게 흐르는지** 그림으로 그립니다.

위에서 아래로 흐릅니다.

1. `config.yaml` + `.env` (키) → **설정/로깅 로더**(29강)
2. 사용자(질문) + `thread_id` → `create_agent` (prompt: 역할/규칙)
3. 에이전트가 아래 구성요소를 골라 사용:

구성요소	데이터 소스
<code>get_order_status</code>	orders.csv
<code>get_stock</code>	inventory.csv
<code>search_faq</code>	faq.csv
<code>policy_search</code>	FAISS (정책 PDF)
<code>InMemorySaver</code>	<code>thread_id</code> 별 대화 기억

4. → 최종 답변 → (선택) FastAPI: `POST /chat {thread_id, message}` (29강)

이 그림이 **30강**에서 구현할 시스템의 청사진입니다. 읽는 법:

- 위에서 아래로: 설정 로드 → 에이전트 생성 → 도구/RAG 실행 → 답변.
- 가운데 `create_agent` 가 지휘자: 질문을 보고 **어떤 도구/RAG**를 쓸지 LLM이 결정(5강 함수 호출).
- 각 도구는 자기 데이터 소스로 연결: 주문은 `orders.csv`, 정책은 FAISS 인덱스.
- `InMemorySaver`: `thread_id` 별로 대화를 기억(19강).

2. 아키텍처가 보여주는 것

좋은 아키텍처 그림은 한눈에 답을 줍니다.

- 데이터가 어디서 와서 어디로 가나? (orders.csv → 도구 → 답변)
- 각 구성요소의 책임은? (도구=조회, RAG=문서검색, 메모리=맥락)
- 어디가 외부와 만나나? (위: 설정, 아래: FastAPI 엔드포인트)
- 무엇이 무엇에 의존하나? (도구는 CSV에, policy_search는 FAISS에)

코드를 짜기 전에 이 그림으로 팀과 합의하고, 빠진 게 없는지 점검합니다. 그림에서 안 보이는 흐름은 코드에서도 영킵니다.

3. 5단계 — 인터페이스 설계 (외부 계약)

에이전트를 서비스로 쓰려면, 외부에서 어떻게 부를지 계약(인터페이스) 을 먼저 정합니다.

항목	설계 결정	이유
진입 함수	<code>answer(message, thread_id) -> str</code>	한 줄로 호출 가능한 단일 진입점
세션 식별	<code>thread_id</code> (사용자/대화 단위)	메모리 격리(멀티 사용자)
상태 저장	<code>InMemorySaver</code> (데모) / 운영은 DB-Redis	프로세스 재시작 대비
엔드포인트	<code>POST /chat {thread_id, message}</code>	무상태 HTTP, 세션은 thread_id로
설정	<code>config.yaml</code> + <code>.env</code> 분리	코드 수정 없이 환경 전환
실패 대비	타임아웃·재시도·폴백 메시지	LLM/네트워크 불안정(29강)
비용·지연	모델 선택·프롬프트 길이·캐싱	무료티어 한도·응답속도

4. 핵심 결정 — 단일 진입점 `answer()`

가장 중요한 인터페이스 결정입니다.

```
def answer(message: str, thread_id: str = "demo") -> str:
    """외부에 노출되는 단일 진입점. thread_id로 대화 세션을 구분한다."""
    ...
```

왜 단일 진입점인가?

[잘된 설계] 에이전트 본체 = 순수 함수 `answer()`
FastAPI 웹 계층 = `answer()`를 부르는 '얇은 래퍼'
→ 테스트(함수 직접 호출) · 재사용 · 배포가 쉬움

[잘못된 설계] 웹 핸들러 안에 에이전트 로직이 다 들어감
→ 테스트하려면 서버를 띄워야 하고, 재사용 불가

에이전트 본체는 순수 함수로 만들고, FastAPI 같은 웹 계층은 이 함수를 부르는 **얇은 껍데기**로 둡니다(29강). 그래야 `answer("안녕")`처럼 **함수만 따로 테스트**할 수 있죠.

5. 세션 격리 — `thread_id`

```
answer("환불 규정?", thread_id="user_123")  
answer("아까 그거 다시", thread_id="user_123") # 같은 thread → 맥락 이어짐  
answer("안녕하세요", thread_id="user_456")      # 다른 thread → 별개 대화
```

`thread_id`로 **사용자/대화를 격리**합니다. 같은 `thread_id`는 같은 대화로 묶여 "아까 그거"가 통과(R5), 다른 `thread_id`는 서로 영향을 안 줍니다. 여러 사용자가 동시에 써도 대화가 안 섞이죠(메모리).

6. 설계 단계에서 미리 챙기는 운영 고려사항

5단계에서 **비용·지연·실패**까지 미리 정해둡니다(상세 구현은 29강).

- 실패 대비: LLM 호출이 타임아웃/rate limit 나면? → 재시도 + 폴백 메시지
- 비용: 호출당 토큰 · 모델 선택 → 무료티어 한도 안에서
- 지연: 프롬프트가 너무 길면 느림 → 간결하게 + (반복 질문) 캐싱
- 설정 분리: `config.yaml`(공개 설정) + `.env`(비밀 키) → 코드 안 고치고 환경 전환

이걸 **설계 단계에서 명시**해두면, 구현할 때 "아 이것도 해야 하네"로 허둥대지 않습니다. 운영의 절반은 설계에서 결정됩니다.

핵심 정리

- **4단계(아키텍처)**: 확정한 구성요소가 **어떻게 연결·흐르는지** 전체 그림으로 그린다 — 30장 청사진.

- 5단계(인터페이스): 외부 계약을 정한다 — 진입 함수·세션·엔드포인트·설정·실패·비용.
- 핵심 결정: 단일 진입점 `answer(message, thread_id)` — 본체는 순수 함수, 웹은 얇은 래퍼.
- `thread_id`로 세션 격리(멀티 사용자 대화가 안 섞임).
- 비용·지연·실패 대비를 설계 단계에서 미리 명시

따라하기 — 설계를 코드 스켈레톤으로

목표

지금까지의 설계를 코드 스켈레톤(함수 시그니처 + TODO 주석)으로 박제합니다. 실제 구현은 30강에서 채우고, 이번엔 실행하면 설계 요약물 출력하는 "살아있는 설계 문서"를 만듭니다. 실제 파일은 `code/ch28_design.py` 입니다.

0. 실습 준비

```
conda activate agentic

# 이번 강은 LLM을 호출하지 않습니다. 설계를 코드로 박제만 합니다.
pip install pyyaml          # (30강 config 로드 대비, 표준 dataclass만 써도 됨)
```

이번 실습은 LLM 호출이 없습니다. `dataclass` 와 함수 시그니처로 설계를 고정하고, 실행하면 설계 요약을 출력하는 게 전부입니다.

1. 시작부 & 설계 명세

`code/ch28_design.py` 를 만듭니다.

```
# -*- coding: utf-8 -*-
# 실행: python code/ch28_design.py
import sys, pathlib
sys.path.append(str(pathlib.Path(__file__).resolve().parent)) # code/ 를 import 경로에
from common import DATA, DOCS
```

```

from dataclasses import dataclass

# [왜 설계 먼저] 30강에서 RAG+도구+메모리를 한꺼번에 구현하기 전에, 이 파일로
# '무엇을 만들지'를 코드 스켈레톤(시그니처+TODO)으로 못 박는다. 함수 이름 · 시그니처를
# 먼저 합의해두면 구현 단계에서 설계가 흔들리지 않는다('살아있는 설계 문서').

@dataclass
class AgentSpec:
    """통합 CS 에이전트의 설계 명세(요약). dataclass 로 설정값을 한눈에 본다."""
    name: str = "승승장구물 통합 CS 에이전트"
    tools: tuple = ("get_order_status", "get_stock", "search_faq", "policy_search")
    use_rag: bool = True # 정책/멤버십 PDF 검색에 RAG 사용
    use_memory: bool = True # 단기 기억(thread 별 대화 세션)
    use_multi_agent: bool = False # [왜 단일] 도구가 4개뿐이라 역할 분리 불필요 → 단일로 시작

```

`AgentSpec` 은 3단계의 판단 결과를 코드로 옮긴 것입니다. "RAG 쓴다(True), 멀티에이전트 안 쓴다(False)"가 한 곳에 박제돼, 나중에 "왜 이렇게 정했지?"의 답이 코드에 남습니다.

2. 도구 시그니처

2단계에서 분해한 함수들을 시그니처 + **TODO** 로 적습니다.

```

def get_order_status(order_id: str) -> str:
    """[R2] 주문번호로 주문 상태를 반환. (30강: orders.csv 조회)"""
    raise NotImplementedError("30강에서 구현") # TODO

def get_stock(product_name: str) -> str:
    """[R3] 상품명으로 재고 수량을 반환. (30강: inventory.csv 조회)"""
    raise NotImplementedError("30강에서 구현") # TODO

def search_faq(keyword: str) -> str:
    """[R4] 키워드로 FAQ 답변을 반환. (30강: faq.csv 조회)"""
    raise NotImplementedError("30강에서 구현") # TODO

def policy_search(query: str) -> str:
    """[R1] 정책/멤버십 문서를 RAG로 검색. (30강: FAISS retriever)"""
    raise NotImplementedError("30강에서 구현") # TODO

```

핵심 기법:

- 독스트링에 요구사항 번호([R1]~[R4]) 를 적어, 함수가 어떤 요구사항을 푸는지 추적 가능.
- 본문은 `raise NotImplementedError("30강에서 구현")` — "아직 안 만들었다"를 명시적으로 표시. 실수로 호출하면 바로 에러로 알려줍니다.
- 시그니처(이름·입력·출력)는 확정 — 30강에서 본문만 채우면 됩니다.

3. 에이전트 빌더 & 진입 함수 시그니처

5단계 인터페이스(`answer`)와 빌더를 적습니다.

```
def build_agent():
    """설정+도구+RAG+메모리를 결합해 create_agent를 만든다. (30강)"""
    # TODO(30강): config 로드 → llm → tools=[...] → checkpoint=InMemorySaver()
    # [설계 의도] InMemorySaver(단기 메모리) + thread_id 로 대화 세션을 구분한다.
    raise NotImplementedError("30강에서 구현")

def answer(message: str, thread_id: str = "demo") -> str:
    """외부에 노출되는 단일 진입점. thread_id로 대화 세션을 구분한다.

    [왜 단일 진입점] FastAPI 같은 외부 호출은 이 함수 하나만 부르면 되게 만든다(29강 운영 계약).
    같은 thread_id = 같은 대화로 묶여 '아까 그 정책' 같은 맥락이 이어진다.
    """
    # TODO(30강): agent.invoke({"messages":[...]}, {"configurable":{"thread_id":...}})
    raise NotImplementedError("30강에서 구현")
```

TODO 주석에 "무엇을 어떻게 구현할지" 까지 적어둡니다(config 로드 → llm → tools → checkpoint). 배포(deployment)를 위해 자세한 메모를 남기는 거죠.

4. 설계 요약 출력 — 살아있는 문서

실행하면 설계가 보이게 만듭니다.

```
def print_design():
    spec = AgentSpec()
    print("=" * 60)
    print(f"[설계] {spec.name}")
    print("=" * 60)
    print("요구사항:")
    print("  R1 정책 안내(RAG) / R2 주문상태 / R3 재고 / R4 FAQ / R5 대화기억")
    print(f"\n도구 목록: {' '.join(spec.tools)}")
    print(f"RAG 사용 : {spec.use_rag} (대상: 정책/멤버십 PDF @ {DOCS})")
    print(f"메모리   : {spec.use_memory} (단기, thread_id 별)")
    print(f"멀티에이전트: {spec.use_multi_agent} (도구 4개 → 단일 에이전트로 시작)")
    print("\n데이터 흐름:")
    print("  사용자 → answer(message, thread_id) → create_agent")
    print("          → [도구/RAG 선택 실행] → [InMemorySaver 기록] → 답변")
    print(f"\n데이터 소스: {DATA}")
    print("운영 계약 : POST /chat {thread_id, message} (FastAPI, 29강)")

if __name__ == "__main__":
    print_design()
```

실행 결과

[설계] 승승장구몰 통합 CS 에이전트

요구사항:

R1 정책 안내(RAG) / R2 주문상태 / R3 재고 / R4 FAQ / R5 대화기억

도구 목록: get_order_status, get_stock, search_faq, policy_search

RAG 사용 : True (대상: 정책/멤버십 PDF @ .../data/docs)

메모리 : True (단기, thread_id 별)

멀티에이전트: False (도구 4개 → 단일 에이전트로 시작)

데이터 흐름:

사용자 → answer(message, thread_id) → create_agent

→ [도구/RAG 선택 실행] → [InMemorySaver 기록] → 답변

데이터 소스: .../data

운영 계약 : POST /chat {thread_id, message} (FastAPI, 29강)

5. "살아있는 설계 문서"의 가치

이 스켈레톤이 그냥 메모와 다른 점:

[죽은 문서] 워드/노선에 적은 설계 → 코드와 따로 놓 → 점점 안 맞아짐
[살아있는 문서] 코드 자체가 설계 → 실행하면 요약 출력 → 30강에서 TODO만 채움
→ 설계와 구현이 '같은 파일'에서 이어짐

관찰 포인트

- 설계가 코드로 박제돼, 30강에서 시그니처를 그대로 채우면 됩니다.
- `AgentSpec` 한 곳에 "무엇을 쓰는지"가 모여, 의사결정 근거가 남습니다.
- 실행만 해도 설계 요약이 나오니, 팀원에게 공유하기 좋습니다.
- 다음 절(06번)에서는 이 "구성요소 판단"을 규칙으로 자동화한 도우미를 만들어, 다른 프로젝트에도 적용해 봅니다.

핵심 정리

- 설계를 코드 스켈레톤(`@dataclass AgentSpec + 함수 시그니처 + TODO`)으로 박제.
- 도구 본문은 `raise NotImplementedError("30강에서 구현")` — "아직 안 만듦"을 명시.
- 독스트링에 요구사항 번호([R1]~), TODO에 구현 방법 메모 → 30강의 나를 위한 안내.
- `print_design()`으로 실행하면 설계 요약 출력 = 살아있는 설계 문서.
- 설계와 구현이 같은 파일에서 이어져, 배로에선 TODO만 채우면 됨.

트레이드오프 판단 (RAG·멀티에이전트 도입 기준)

03번에서 "구성요소를 기준으로 판단한다"를 배웠습니다. 이걸 규칙으로 자동화한 설계 도우미를 만듭니다.

입력 신호(도구 수·사내문서·실시간데이터·멀티턴·역할 수)를 점수화해, RAG·장기메모리·멀티에이전트·워크플로우의 도입 여부를 근거와 함께 추천합니다.

LLM 없이 경험칙(규칙) 만으로요.

1. 왜 규칙으로 자동화하나

03번의 판단은 사람이 표를 보고 직접 했습니다. 그런데 프로젝트가 여러 개라면? 매번 "이건 RAG 필요할까?"를 고민하는 대신, 판단 기준을 코드로 박아두면 일관되게 추천받을 수 있습니다.

[매번 사람이 판단] 프로젝트마다 기준이 흔들림 → "지난번엔 왜 RAG 넣었지?"
[규칙으로 자동화] 같은 입력 → 같은 추천 + 근거 → 일관성 · 재현성

왜 LLM이 아니라 규칙?

설계 초반 의사결정은 LLM 없이도 **경험칙(규칙)** 으로 충분히 잡힙니다.

"사내문서 있으면 RAG", "역할 3개+면 멀티에이전트" 같은 건 명확한 규칙이죠.

매번 LLM에 묻는 건 비용·불일치만 늘립니다. (규칙 라우터의 정신.)

2. 설계 입력 명세

판단에 필요한 신호를 `dataclass` 로 정의합니다(28강 `AgentSpec` 스타일).

```
from dataclasses import dataclass

@dataclass
class DesignSpec:
    """설계 판단에 필요한 입력 신호."""
    name: str = "이름없는 에이전트"
    num_tools: int = 0          # 불일 도구 개수
    has_internal_docs: bool = False # 사내문서(정책·매뉴얼 PDF 등) 유무
    needs_realtime: bool = False  # 실시간 데이터(주문·재고) 필요 여부
    multi_turn: bool = False      # 멀티턴(대화 맥락 유지) 필요 여부
    num_roles: int = 1           # 분리할 역할 수(리서처/분석가/작성자 등)
    long_running: bool = False   # 장기 누적 기억(사용자별 프로필) 필요 여부
```

이 7개 신호만 입력하면, 도우미가 구성요소 도입 여부를 추천합니다.

3. 판단 로직 — 점수와 근거

각 구성요소를 점수로 평가하고, 임계값을 넘으면 "도입"으로 추천합니다. 핵심은 근거를 함께 남기는 것입니다.

```
def decide(spec: DesignSpec) -> dict:
    """입력 신호를 규칙·점수로 평가해 구성요소 도입 여부와 근거를 반환한다."""
```

```

result = {}

# --- RAG: 사내문서가 있으면 강하게 권장 ---
rag_score, rag_reasons = 0, []
if spec.has_internal_docs:
    rag_score += 2
    rag_reasons.append("사내문서 보유 → 근거 있는 답변 위해 RAG 필요")
if spec.needs_realtime:
    rag_reasons.append("단, 실시간 데이터는 RAG가 아니라 도구로 조회")
result["RAG"] = {"도입": rag_score >= 2, "점수": rag_score,
                 "근거": " / ".join(rag_reasons) or "검색할 고정 문서가 없어 불필요"}

# --- 장기메모리: 멀티턴 + 장기누적이면 권장 ---
mem_score, mem_reasons = 0, []
if spec.multi_turn:
    mem_score += 1
    mem_reasons.append("멀티턴 → 최소 단기메모리(thread_id) 필요")
if spec.long_running:
    mem_score += 2
    mem_reasons.append("사용자별 장기 누적 → 장기메모리(프로필 저장) 권장")
result["장기메모리"] = {"도입": mem_score >= 2, "점수": mem_score,
                       "근거": " / ".join(mem_reasons) or "단발성 대화라 메모리 불필요"}

# --- 멀티에이전트: 역할 수 · 도구 수가 많으면 권장 ---
ma_score, ma_reasons = 0, []
if spec.num_roles >= 3:
    ma_score += 2
    ma_reasons.append(f"역할 {spec.num_roles}개 → 역할별 에이전트 분리 권장")
if spec.num_tools >= 8:
    ma_score += 1
    ma_reasons.append(f"도구 {spec.num_tools}개 → 단일 에이전트엔 과부하")
result["멀티에이전트"] = {"도입": ma_score >= 2, "점수": ma_score,
                          "근거": " / ".join(ma_reasons) or "역할 · 도구가 적어 단일로 시작"}

# --- 워크플로우(StateGraph): 역할이 정해진 순서로 흐르면 권장 ---
wf_score, wf_reasons = 0, []
if spec.num_roles >= 2:
    wf_score += 2
    wf_reasons.append(f"역할 {spec.num_roles}개의 정해진 처리 순서 → 워크플로우로 명시")
result["워크플로우"] = {"도입": wf_score >= 2, "점수": wf_score,
                       "근거": " / ".join(wf_reasons) or "단일 역할이라 선형 호출로 충분"}
return result

```

설계 포인트

- 각 구성요소가 자기 규칙으로 점수를 매김(사내문서 → RAG +2, 역할 3개+ → 멀티에이전트 +2).
- 임계값(>= 2)을 넘으면 도입 추천.
- 근거를 항상 반환 — "왜 도입/보류인지"가 남아야 신뢰할 수 있는 추천.
- RAG 규칙에 "실시간은 RAG 아님(도구)" 주의를 넣어, 03번의 헛갈리는 구분을 자동으로 짚어줌.

4. 출력과 실행

추천 결과를 보기 좋게 출력합니다.

```
def print_decision(spec: DesignSpec) -> None:
    print("=" * 60)
    print(f"[시나리오] {spec.name}")
    print(f"  도구={spec.num_tools}개, 사내문서={spec.has_internal_docs}, "
          f"  실시간={spec.needs_realtime}, 멀티턴={spec.multi_turn}, "
          f"  역할={spec.num_roles}개, 장기={spec.long_running}")
    print("-" * 60)
    for i, (comp, d) in enumerate(decide(spec).items(), 1):
        mark = "도입 ✅" if d["도입"] else "보류 ❌"
        print(f"[{i}] {comp:<7} {mark} (점수 {d['점수']}) — {d['근거']}")
```

여러 시나리오로 돌려봅니다.

```
if __name__ == "__main__":
    scenarios = [
        DesignSpec(name="승승장구물 통합 CS 에이전트",
                    num_tools=4, has_internal_docs=True, needs_realtime=True,
                    multi_turn=True, num_roles=1, long_running=False),
        DesignSpec(name="승승장구물 시장조사 팀",
                    num_tools=2, has_internal_docs=False, needs_realtime=False,
                    multi_turn=False, num_roles=3, long_running=False),
        DesignSpec(name="승승장구물 간단 FAQ 봇",
                    num_tools=1, has_internal_docs=False, needs_realtime=False,
                    multi_turn=False, num_roles=1, long_running=False),
        DesignSpec(name="승승장구물 VIP 개인비서",
                    num_tools=10, has_internal_docs=True, needs_realtime=True,
                    multi_turn=True, num_roles=2, long_running=True),
    ]
    for sp in scenarios:
        print_decision(sp); print()
```

예상 출력 (일부)

[시나리오] 승승장구물 통합 CS 에이전트

도구=4개, 사내문서=True, 실시간=True, 멀티턴=True, 역할=1개, 장기=False

- [1] RAG 도입 ☒ (점수 2) — 사내문서 보유 → 근거 있는 답변 / 단, 실시간은 도구로 조회
- [2] 장기메모리 보유 ☐ (점수 1) — 멀티턴 → 최소 단기메모리(thread_id) 필요
- [3] 멀티에이전트 보유 ☐ (점수 0) — 역할 · 도구가 적어 단일로 시작
- [4] 워크플로우 보유 ☐ (점수 0) — 단일 역할이라 선형 호출로 충분

[시나리오] 승승장구물 시장조사 팀

도구=2개, ..., 역할=3개, ...

- [1] RAG 보유 ☐ (점수 0) — 검색할 고정 문서가 없어 불필요
- [3] 멀티에이전트 도입 ☒ (점수 2) — 역할 3개 → 역할별 에이전트 분리 권장
- [4] 워크플로우 도입 ☒ (점수 2) — 역할 3개의 정해진 처리 순서 → 워크플로우로 명시

5. 결과 읽기 — 케이스별 추천이 다르다

같은 도우미가 시나리오마다 다른 추천을 냅니다.

통합 CS 에이전트: RAG ☒ (사내문서) / 멀티에이전트 ☐ (도구 4개) → 28강 설계와 일치!
시장조사 팀: RAG ☐ / 멀티에이전트 ☒ + 워크플로우 ☒ (역할 3개) → 27강 AI팀과 일치!
간단 FAQ 봇: 전부 ☐ → 도구 1개로 충분, 과한 구성 불필요
VIP 개인비서: RAG ☒ + 장기메모리 ☒ (long_running) + 멀티에이전트 보유 → 균형

우리가 **project design**에서 손으로 내린 판단(RAG ☒, 멀티에이전트 ☒)을 규칙이 그대로 재현합니다. 즉 이 도우미는 03번의 판단 기준을 코드로 옮긴 것이고, 다른 프로젝트에도 일관되게 적용할 수 있습니다.

⚠ **한계** — 이건 출발점이지 정답이 아닙니다. 규칙은 경험칙이라, 경계 사례(예: 도구 7개)에선 사람이 최종 판단해야 합니다. 도우미는 "생각의 체크리스트"로 쓰고, 결정은 사람이 내립니다.